

Seed = 1

Data:

Imię i Nazwisko:

Numer Indeksu:

Systemy Sztucznej Inteligencji

Za każde zadanie można otrzymać 1 punkt, jeśli zaznaczone zostały wszystkie poprawne odpowiedzi. Brak punktów w przypadku zaznaczenia błędnej odpowiedzi lub niepełnego zaznaczenia poprawnych opcji.

1) Które stwierdzenia mają charakter *behawioralnej* definicji SI (ocena po efektach)?

☐ O przynależności do SI decyduje tylko język programowania.

☒ System uznajemy za inteligentny, jeśli jego zachowanie jest nieodróżnialne od ludzkiego w danym zadaniu.

☒ Racjonalność oceniana jest względem funkcji użyteczności/miary osiągnięć.

☐ Wymagana jest świadomość maszyny, inaczej nie jest to SI.

☐ SI definiuje wyłącznie wewnętrzna implementacja, a nie wyniki.

2) Zastosowania SI w *energetyce* obejmują:

☐ Tworzenie darmowej energii z próżni dzięki głębokim sieciom.

☐ Ominięcie praw Kirchhoffa przez strojenie hiperparametrów.

☐ Zamianę prądu zmiennego w stały bez przekształtników — samym algorytmem.

☒ Wykrywanie anomalii i przewidywanie awarii w sieciach przesyłowych.

☒ Prognozowanie obciążenia sieci i produkcji OZE.

3) Jaki jest sens eksperymentu myślowego „*chiński pokój*” (Searle’a) w dyskusji o definicji SI?

☐ Teza, że rozumienie wynika wyłącznie z dużej mocy obliczeniowej (FLOPS).

☒ Ma pokazać, że czysta manipulacja symbolami może wystarczyć do zdań „inteligentnych” na zewnątrz, ale nie implikuje *rozumienia*.

☐ Przykład, że logika rozmyta zawsze daje poprawne tłumaczenia zdań.

☐ To dowód matematyczny, że żadna maszyna nie może przejść testu Turinga.

☐ Procedura inżynierska kalibracji tłumacza maszynowego dla języka chińskiego.

4) Które przykłady opisują *pętlę percepcja–działanie*?

☒ Robot lokalizuje przeszkodę na obrazie i hamuje, by uniknąć kolizji.

☐ Kalkulator wykonujący $2 + 2$ na żądanie bez stanu i strategii.

☐ Archiwizator ZIP kompresujący folder bez żadnych decyzji.

☐ Skrypt, który drukuje „Hello World” bez wejścia.

☒ Agent giełdowy przetwarza strumień notowań i aktualizuje portfel zgodnie z polityką.

5) Przyporządkuj przykłady do paradygmatów: *nadzorowane*, *nienadzorowane*, *samonadzorowane*, *RL*.

☒ Policy gradient / Q-learning → uczenie ze wzmocnieniem (RL).

☒ Maskowane modelowanie języka (MLM) → uczenie samonadzorowane.

☒ K-means / klasteryzacja cech → uczenie nienadzorowane.

☒ Klasyfikacja obrazów z etykietami → uczenie nadzorowane.

☒ Autoenkoder rekonstrukcyjny → uczenie nienadzorowane/samonadzorowane.

6) Które przykłady obrazują *szkodę* potencjalnie wywołaną przez systemy SI?

☐ Obniżenie zużycia energii centrum danych dzięki optymalizacji chłodzenia.

☒ Dyskryminacja grup chronionych w decyzjach kredytowych.

☒ Błędne rozpoznanie tożsamości w systemach nadzoru prowadzące do zatrzymań.

☐ Zwiększenie trafności rekomendacji przy pełnym poszanowaniu prywatności.

☐ Przejrzyste raporty metryk publikowane po wdrożeniu.

7) Które *obszary i kompetencje* sensownie mieszczą się w definicyjnym zakresie SI?

☒ Planowanie i podejmowanie decyzji.

☒ Przeszukiwanie i optymalizacja heurystyczna.

☒ Uczenie maszynowe.

☒ Percepcja i integracja sensoryczna.

☒ Reprezentacja wiedzy i wnioskowanie.

8) W jakim celu agent utrzymuje *model świata*?

- ☐ Ponieważ model świata jest wymagany nawet w środowiskach statycznych bez dynamiki.
- ☐ Wyłącznie po to, by przechowywać zrzuty ekranu GUI.
- Aby przewidywać efekty akcji i planować sekwencje działań przed ich wykonaniem.
- Aby lepiej radzić sobie z niepewnością i częściową obserwowalnością.
- ☐ Aby zastąpić percepcję losowym zgadywaniem.

9) Definicja *agenta racjonalnego* w SI mówi, że agent:

- ☐ Ignoruje informację zwrotną o skutkach swoich działań.
- ☐ Zawsze działa losowo, aby uniknąć uprzedzeń.
- ☐ Maksymalizuje zużycie CPU jako miarę „inteligencji”.
- ☐ Nie posiada żadnej reprezentacji świata ani pamięci.
- Wybiera działania maksymalizujące oczekiwaną miarę osiągnięć na podstawie percepcji, wiedzy i funkcji celu.

10) Co zwykle odróżnia *wymagania danych i mocy obliczeniowej* DL od klasycznego ML?

- ☐ DL jest z natury *łżejsze* obliczeniowo od drzew decyzyjnych w każdej skali.
- Czas trenowania modeli DL bywa rzędu wielkości dłuższy niż w klasycznym ML.
- Przy małych danych często lepiej sprawdzają się modele prostsze lub transfer learning.
- DL zazwyczaj wymaga większych zbiorów danych i większej mocy obliczeniowej.
- Regularizacja i augmentacja danych są kluczowe w DL, by ograniczyć przeuczenie.

11) Które stwierdzenia o *uczeniu ze wzmocnieniem (RL)* jako paradygmacie są poprawne?

- Agent maksymalizuje skumulowaną nagrodę.
- ☐ W RL nie występuje eksploracja–eksploatacja.
- ☐ RL nie korzysta z funkcji nagrody; decyzje są losowe.
- ☐ RL wymaga pełnych etykiet klas dla każdego stanu jak w klasycznej klasyfikacji.
- Polityka (policy) określa wybór akcji na podstawie stanu.

12) *Paradygmat hybrydowy* (neuro-symboliczny) dąży do:

- Połączenia uczenia konekjonistycznego z wnioskowaniem symbolicznym.
- ☐ Eliminacji zarówno danych, jak i reguł.
- ☐ Zastąpienia wszystkich wag sieci neuronowej stałymi logicznymi.
- Lepszej interpretowalności i wykorzystania wiedzy domenowej w modelach uczących się.
- ☐ Zabronienia użycia probabilistyki w jakiegokolwiek formie.

13) Jak najlepiej opisać relacje między *AI*, *ML* i *DL*?

- ☐ AI = jedynie systemy regułowe, bez statystyki i danych.
- ☐ ML zawsze wymaga etykietowanych danych — w przeciwnym razie nie jest ML.
- DL realizuje *uczenie reprezentacji*, często redukując potrzebę ręcznej inżynierii cech.
- ML może obejmować metody inne niż sieci neuronowe.
- AI to parasol pojęciowy; ML jest podzbiorem AI uczącym się z danych; DL jest podzbiorem ML opartym na sieciach neuronowych z wieloma warstwami.

14) Wyjaśnialność (XAI) jest etycznie istotna, ponieważ:

- ☐ Pozwala pominąć zgodność z przepisami.
- ☐ Gwarantuje, że model nie popełni żadnego błędu.
- ☐ Zastępuje konieczność testów jakości i bezpieczeństwa.
- Wspiera rozliczalność wobec użytkowników i regulatorów.
- Ułatwia zrozumienie podstaw decyzji i identyfikację potencjalnych błędów lub uprzedzeń.

15) Które działania są zgodne z *zasadą minimalizacji szkody* przy projektowaniu SI?

- Włączenie zespołów ds. bezpieczeństwa i etyki na wczesnym etapie projektu.
- Testy bezpieczeństwa, ocena ryzyka i plany reakcji na incydenty.
- ☐ Wyłączanie mechanizmów kontroli dostępu, by przyspieszyć eksperymenty.
- ☐ Stosowanie ciemnych wzorców w interfejsie.
- ☐ Publikowanie danych wrażliwych w repozytoriach publicznych.

16) Wskaż stwierdzenia, które *NIE* należą do definicyjnego zakresu sztucznej inteligencji.

- Renderowanie grafiki 4K i animacja komputerowa.
- Programowanie obiektowe jako jedyne kryterium „inteligencji”.
- ☐ Automatyzacja rozumowania, uczenia się i percepcji w celu osiągnięcia zdefiniowanych celów.
- Zwiększanie częstotliwości CPU bez zmiany algorytmów.
- ☐ Projektowanie agentów racjonalnych podejmujących decyzje na podstawie percepcji, wiedzy i funkcji celu.

17) W definicjach SI często występuje *funkcja celu*. Które przykłady są zgodne z takim ujęciem?

- Minimalizacja błędów decyzyjnych w klasyfikacji.
- ☐ Maksymalizacja zużycia pamięci RAM jako miary „inteligencji”.
- ☐ Losowa zmiana parametrów bez kryterium oceny.
- ☐ Minimalizacja rozmiaru pliku PDF niezależnie od jakości decyzji.
- Maksymalizacja skumulowanej nagrody w uczeniu ze wzmocnieniem.

18) *Działanie* (akcja) w pętli agent–środowisko:

- ☐ Musi być liczbą rzeczywistą — akcje dyskretne są niedozwolone.
- Jest wyborem agenta, który wpływa na stan środowiska i przyszłe obserwacje.
- ☐ Występuje tylko w systemach bez sprzężenia zwrotnego.
- ☐ Zawsze jest równoznaczne z odczytem sensora.
- ☐ Nie ma wpływu na nagrody/miarę osiągnięć.

19) Jakie własności *środowiska* są kluczowe przy projektowaniu agenta?

- ☐ Kolor interfejsu użytkownika w aplikacji webowej.
- Pełna vs. częściowa obserwowalność.
- Deterministyczne vs. stochastyczne przejścia stanów.
- Ciągłe vs. dyskretne przestrzenie stanów i akcji.
- ☐ Liczba rdzeni CPU komputera.

20) Które *wymagania niefunkcjonalne* są kluczowe dla komponentu wdrożeniowego AI?

- ☐ Wyłącznie zgodność z motywem kolorystycznym aplikacji.
- Skalowalność, niezawodność i bezpieczeństwo danych.
- Opóźnienie i przepustowość predykcji.
- ☐ Tylko rozmiar pliku modelu, bez wpływu na QoS.
- ☐ Losowe restartowanie usługi celem „odświeżenia” wag.

21) Gdzie rozsądnie stosuje się SI w *transporcie i mobilności*?

- ☐ Teleportację ładunków z użyciem sieci neuronowych.
- ☐ Zastąpienie wszystkich sygnalizacji świetlnych losowym sterowaniem.
- ☐ Legalne prowadzenie pojazdów autonomicznych bez nadzoru w każdym kraju i warunkach.
- Zaawansowane systemy wspomagania kierowcy.
- Optymalizacja tras i harmonogramów flot.

22) *Środowisko* w modelu agent–środowisko:

- Dostarcza agentowi obserwacji i przyjmuje jego działania, ewoluując według swoich reguł.
- ☐ To wyłącznie zbiór etykiet klas w uczeniu nadzorowanym.
- ☐ Jest z definicji w pełni kontrolowane przez agenta.
- ☐ Nie ma żadnej dynamiki; stan jest zawsze stały.
- Może być deterministyczne lub stochastyczne, obserwowalne częściowo lub w pełni.

23) Które z poniższych najlepiej oddaje ogólną definicję *sztucznej inteligencji (SI)*?

- ☐ Standard graficzny do renderowania scen 3D.
- ☐ Wyłącznie roboty humanoidalne wyposażone w czujniki i aktuatory.
- Dziedzina informatyki tworząca systemy zdolne wykonywać zadania zwykle wymagające ludzkiej inteligencji.
- ☐ Synonim programowania obiektowego i wzorców projektowych.
- ☐ Zbiór reguł do kompresji danych i szyfrowania informacji.

24) W jakich sytuacjach *klasyczny ML* może być lepszym wyborem niż DL?

- Silny wymóg interpretowalności (np. modele liniowe, małe drzewa).
- Mały zbiór danych tabularnych, potrzeba szybkiego treningu i prostego wdrożenia.
- Gdy cechy są dobrze zdefiniowane domenowo i ich liczba jest niewielka.
- ☐ Gdy chcemy łączyć wiele modalności (obraz+tekst+audio) bez inżynierii cech.
- ☐ Gdy dane to surowe obrazy 4K i nagrania audio — wtedy ML zwykle przewyższa DL.

25) Które działania wspierają *odpowiedzialne wdrażanie* SI w organizacji?

- ☐ Wyłączanie monitoringu jakości.
- ☐ Brak dokumentacji modeli i danych.
- ☐ Ukrywanie incydentów i błędów przed użytkownikami.
- Wyznaczenie ról i właścicieli procesu oraz prowadzenie rejestru modeli.
- Ocena ryzyka przed wdrożeniem.

26) Które przykłady to *aktuatory* (kanały działania) w systemach agentowych?

- ☐ Zbiór walidacyjny $\mathcal{D}_{val}$.
- ☐ Kamera RGB-D i lidar.
- Silniki kół, serwomechanizmy ramion, głośniki do generowania komunikatów.
- Wywołania API sterujące systemem.
- ☐ Plik `requirements.txt`.

27) Jaki jest sens *testu Turinga* w kontekście definiowania SI?

- ☐ To procedura kalibracji czujników w robotyce mobilnej.
- System uznaje się za „inteligentny”, jeśli w rozmowie tekstowej kompetentny sędzia nie potrafi odróżnić go od człowieka.
- ☐ To definicja semantyki języka programowania Prolog.
- ☐ To benchmark GPU do renderingu w czasie rzeczywistym.
- ☐ To test złożoności obliczeniowej algorytmu w notacji $\mathcal{O}(\cdot)$.

28) W jakim celu definiuje się *funkcję straty* $\mathcal{L}(\theta)$ w systemie AI?

- ☐ Aby zdefiniować format zapisu plików `.csv`.
- ☐ Aby zaszyfrować dane wejściowe podczas inferencji.
- Aby ilościowo ocenić błąd modelu i kierować optymalizacją parametrów θ .
- Aby umożliwić dobór hiperparametrów przez walidację pod kątem jakości.
- ☐ Aby generować syntetyczne etykiety bez danych.

29) W *sztuce i mediach* SI jest używana do:

- Generowania szkiców koncepcji jako wsparcie twórców.
- Personalizacji rekomendacji treści w serwisach streamingowych.
- ☐ Gwarantowanego zastąpienia wszystkich artystów do 2030 roku.
- ☐ Natychmiastowego tłumaczenia dowolnego wideo bez błędów semantycznych w każdym języku.
- ☐ Legalnego kopiowania cudzych dzieł bez licencji.

30) Które zdania o *ocenieniu modeli* w AI/ML/DL są trafne?

- Dopasowanie metryk skuteczności do zadania.
- ☐ Jedna metryka wystarcza we wszystkich problemach i klasach niezbalansowanych.
- Walidacja krzyżowa bywa trudniejsza w bardzo dużych modelach DL z uwagi na koszt obliczeń.
- ☐ Testujemy algorytm na zbiorze treningowym.
- Należy rozdzielać dane na zbiory: treningowy, walidacyjny i testowy.

31) Które praktyki pomagają ograniczać *stronniczość* (bias) w danych i modelach?

- Równoważenie zbiorów i testy równości metryk między grupami.
- Monitorowanie driftu i testy uczciwości po wdrożeniu.
- ☐ Celowe wykluczanie grup użytkowników z ewaluacji.
- Audyt i dokumentowanie danych.
- ☐ Ukrywanie metryk, aby nie było widać nierówności.

32) (Python) Które polecenia tworzą środowisko?

- ☐ `pip create proj`
- ☐ `proj\Scripts\activate.bat`
- ☐ `.\proj\Scripts\Activate.ps1`
- ☐ `environment -m venv proj`
- ☒ `python -m venv proj`

33) (Python) Jak *wyłączyć* aktywne środowisko `venv`?

- ☐ `python -m venv --off`
- ☒ Zamknięcie/wyjscie z terminala kończy aktywację w tej sesji.
- ☐ `pip deactivate`
- ☒ Polecenie `deactivate` w aktywnej sesji powłoki.
- ☐ `pip uninstall venv`

34) (Python/NumPy) Jak utworzyć wektor `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`?

- ☐ `numpy.arrayrange(0,9)`
- ☐ `numpy.arange(1,11)`
- ☐ `numpy.linspace(0,10)`
- ☐ `numpy.range(10)`
- ☒ `numpy.arange(10)`

35) (Python) Czym jest `pandas`?

- ☐ Samodzielny system bazodanowy SQL uruchamiany jako usługa.
- ☒ Warstwa wysokiego poziomu zbudowana na `NumPy`, ułatwiająca wczytywanie, czyszczenie, łączenie i grupowanie danych.
- ☐ Framework do uczenia ze wzmocnieniem (RL).
- ☒ Biblioteka do analizy danych w Pythonie, oferująca struktury i operacje na danych tabelarycznych.
- ☐ Silnik renderujący wykresy 3D i animacje w czasie rzeczywistym.

36) (Python) Które polecenie *poprawnie aktywuje* środowisko `proj` w Windows?

- ☒ CMD: `proj\Scripts\activate.bat`
- ☒ PowerShell: `.\proj\Scripts\Activate.ps1`
- ☐ `source proj/bin/activate`
- ☐ `pip activate proj`
- ☐ `py -m venv proj`

37) (Python/NumPy) Które zapisy stworzą lub skonwertują tablicę do typu `float`?

- ☒ `numpy.arange(5).astype(float)`
- ☐ `numpy.asarray([1,2,3]).astype(int)`
- ☐ `numpy.arange(5, dtype=int).storeas(float)`
- ☒ `numpy.array([1,2,3], dtype=float)`
- ☐ `numpy.float([1,2,3])`

38) (Python/pandas) Z `pandas.DataFrame df` wybierz kolumny `Name` i `Age` oraz tylko wiersze z `Age > 25`. Które zapisy są poprawne?

- ☐ `df.where("Age > 25")[["Name", "Age"]]`
- ☒ `df.query("Age > 25")[["Name", "Age"]]`
- ☐ `df.select(columns=["Name", "Age"], where="Age > 25")`
- ☒ `df.loc[df["Age"] > 25, ["Name", "Age"]]`
- ☐ `df.iloc[df["Age"] > 25, ["Name", "Age"]]`

39) (Python) Które stwierdzenia poprawnie porównują **listę** i **krotkę (tuple)**?

- ☐ List nie można indeksować ujemnie, a krotek tak.
- ☒ Krotki mogą być używane jako klucze słownika, jeśli ich elementy są hashowalne.
- ☐ Krotka zawsze zajmuje więcej pamięci niż lista o tej samej zawartości.
- ☐ Krotka ma metodę `append`, lista jej nie ma.
- ☒ Lista jest modyfikowalna (mutable), krotka jest niemodyfikowalna (immutable).

40) (Python/NumPy) Które wyrażenia liczą średnią arytmetyczną elementów tablicy `a`?

- ☐ `numpy.average()`
- ☒ `numpy.sum(a) / len(a)`
- ☐ `a.median()`
- ☒ `a.mean()`
- ☒ `numpy.mean(a)`

41) (Python) Które zapisy dadzą wynik `[1, 2, 3, 4, 5]` startując od `lst = [1,2,3]`?

- ☒ `lst += [4,5]; lst`
- ☒ `lst.extend([4,5]); lst`
- ☐ `lst.append([4,5]); lst`
- ☐ `lst = lst.append(4).append(5); lst`
- ☐ `lst.add(4,5); lst`

42) (Python/pandas) Jak odczytać plik CSV `people.csv` do `pandas.DataFrame` i zobaczyć podstawowe statystyki?

- ☐ `pandas.load_csv("people.csv"); df.describe()`
- ☐ `df = pandas.read_csv("people.csv"); df.info()`
- ☐ `df = pandas.DataFrame.read("people.csv"); df.summary()`
- ☒ `df = pandas.read_csv("people.csv"); df.describe()`
- ☐ `df = read_csv("people.csv"); df.describe()`

43) (Python) Które stwierdzenia o operatorze `in` dla `dict` są prawdziwe?

- ☒ `"a" in {"a": 1, "b": 2}` sprawdza obecność *klucza*.
- ☐ `"a" in d.values()` sprawdza klucze, nie wartości.
- ☒ `2 in {"a": 1, "b": 2}` zwraca `False`, bo `in` nie sprawdza wartości.
- ☐ `("a",1) in d` sprawdza pary klucz–wartość.
- ☐ `in` nie działa na słownikach.

44) (Python/pandas) Jak policzyć licznosc grup `pandas.DataFrame df` według `Age` (ile osób w każdym wieku)?

- ☐ `df.groupby("Age").len()`
- ☐ `df.value_counts()`
- ☐ `df.count("Age")`
- ☒ `df.groupby("Age").size()`
- ☒ `df.groupby("Age")["Name"].count()`

45) (Python) Które stwierdzenia o `pop`, `remove`, `clear` są prawdziwe?

- ☐ `lst.pop(0)` usuwa ostatni element (zawsze).
- ☐ `lst.remove(i)` usuwa element na pozycji `i`.
- ☒ `lst.clear()` usuwa wszystkie elementy; `lst` staje się pustą listą.
- ☒ `lst.pop()` usuwa i zwraca ostatni element listy.
- ☒ `lst.remove(x)` usuwa pierwsze wystąpienie wartości równej `x`.

46) (Python/NumPy) Który zapis poprawnie tworzy macierz 2×3 z liczb $0, \dots, 5$?

- ☐ `numpy.array(6).reshape(2,3)`
- ☐ `numpy.reshape(numpy.arange(6), (3,2))`
- ☒ `numpy.arange(6).reshape(2,3)`
- ☐ `numpy.arange(1,7).reshape(2,3)`
- ☐ `numpy.arange(2,3).reshape(6)`

47) (Python) Co zwróci poniższy kod?

- ```
lst = [1,2,3]; lst.append([4,5]); lst
```
- ☐ `[1, 2, 3, 5, 4]`
  - ☒ `[1, 2, 3, [4, 5]]`
  - ☐ Zgłosi błąd typu, bo `append` przyjmuje tylko liczby.
  - ☐ `[[1,2,3], 4, 5]`
  - ☐ `[1, 2, 3, 4, 5]`

48) (Python/NumPy) Jak wybrać tylko liczby parzyste z `a = numpy.arange(10)`?

- ☐ `a.filter(lambda x: x % 2 == 0)`
- ☐ `a[(2)]`
- ☐ `a.where(a % 2 == 0)`
- ☒ `a[a % 2 == 0]`
- ☐ `numpy.select(a, a % 2 == 0)`

49) (Python) Które narzędzia stanowią *alternatywy* lub uzupełnienia dla `venv` w zarządzaniu środowiskami/zależnościami?

- ☒ `virtualenv`
- ☒ `poetry`
- ☐ `npm`
- ☒ `pipenv`
- ☒ `conda` (Anaconda/Miniconda)

50) (Python/NumPy) Jak wylosować 3 liczby z rozkładu  $\mathcal{N}(0, 1)$  (*rozkład normalny*)?

- ☒ `x = numpy.random.default_rng().standard_normal(3)`
- ☐ `x = numpy.random.default_rng().uniform(0, 1, 3)`
- ☒ `x = numpy.random.default_rng().normal(loc=0.0, scale=1.0, size=3)`
- ☐ `x = numpy.random.select(loc=0.0, scale=1.0, size=3)`
- ☐ `x = numpy.standarization().draw_normal(3)`

51) (Python/pandas) Jak utworzyć `pandas.DataFrame` z dwóch list `names`, `ages` z kolumnami `Name`, `Age`?

- ☒ `df = pandas.DataFrame({"Name": names, "Age": ages})`
- ☐ `df = pandas.DataFrame(names, ages)`
- ☐ `df = pandas.read_csv([names, ages])`
- ☐ `df = pandas.Series("Name": names, "Age": ages)`
- ☒ `df = pandas.DataFrame.from_dict({"Name": names, "Age": ages})`

52) Dobierz **właściwe techniki przetwarzania** do typu cechy.

- ☒ Cechy kategoryczne: one-hot encoding.
- ☒ Braki: numeryczne  $\rightarrow$  mediana/średnia; kategoryczne  $\rightarrow$  moda albo specjalna kategoria.
- ☒ Cechy numeryczne: standaryzacja lub skalowanie min-max.
- ☐ Cechy kategoryczne należy zawsze skalować min-max przed one-hot encodingiem.
- ☒ Cechy kategoryczne wysokokardynalne: target/mean encoding.

53) Kiedy **logarytmowanie** (`np. log(x + 1)`) jest uzasadnione?

- ☐ Zawsze dla danych symetrycznych o rozkładzie  $\mathcal{N}(0, 1)$ .
- ☐ Dla liczb ujemnych bez żadnej modyfikacji argumentu.
- ☒ Gdy chcemy zmniejszyć wpływ dużych outlierów na model liniowy.
- ☒ Gdy rozkład jest silnie prawoskośny (długi ogon) i wartości są nieujemne.
- ☐ Zawsze zamiast normalizacji.

54) Które stwierdzenia o **przekleństwie wymiarowości** są prawdziwe?

- ☐ Zawsze lepiej dodać szumowe cechy niż je usunąć.
- ☐ Więcej cech zawsze poprawia generalizację bez dodatkowych danych.
- ☒ Wysoka liczba cech zwiększa rzadkość danych i ryzyko przeuczenia.
- ☐ Problem dotyczy tylko danych kategorycznych po one-hot encoding.
- ☒ Modele odległościowe (k-NN) tracą rozróżnialność przy bardzo wielu wymiarach.

55) Jakie wykresy pomagają **zidentyfikować outliery**?

- ☒ Wykres rozrzutu (scatter) dla par cech.
- ☐ Mapa cieplna korelacji *zawsze* wykrywa outliery punktowe.
- ☐ Histogram *zawsze* ukrywa outliery dzięki binowaniu.
- ☐ Word cloud etykiet kategorycznych.
- ☒ Wykres pudełkowy (boxplot) z wąsami IQR.

56) Które sytuacje wskazują na **braki danych**, wymagające uzupełnienia lub usunięcia rekordów?

- W kolumnie **age** występują wartości NaN.
- ☐ Wartości w **temperature\_C** są liczbami zmiennoprzecinkowymi bez braków.
- ☐ Wszystkie wartości w kolumnie **id** są unikalne liczby całkowite.
- ☐ Kolumna **gender** ma dwie poprawne etykiety: "F", "M" bez pustych pól.
- W pliku CSV puste pola oznaczone są jako "" lub "N/A".

57) Które przykłady wskazują na **duplikaty rekordów** wymagające usunięcia?

- ☐ Zduplikowane nazwy kolumn w schemacie.
- Po połączeniu tabel te same wiersze pojawiają się wielokrotnie.
- ☐ Wiersze mają tę samą nazwę klienta, ale inne adresy.
- ☐ W tabeli występują dwie różne pozycje o tym samym **order\_id**, ale innym **timestamp**.
- W tabeli zamówień rekordy mają ten sam **order\_id** i jednakowy **timestamp**.

58) Które techniki to służą do selekcji cech?

- Lasso (L1) jako model osadzony (embedded).
- Próg wariancji.
- Korelacja Spearmana.
- RFE (Recursive Feature Elimination) z estymatorem.
- Selekcja na podstawie korelacji z celem.

59) **Ordinal encoding** jest właściwy, gdy:

- ☐ Chcemy uniknąć wprowadzania relacji  $<$  między kategoriami.
- ☐ Lepiej zastosować PCA na kategoriach.
- ☐ Kategorie są nominalne bez porządku.
- Kategorie mają naturalny porządek.
- ☐ Zawsze należy go łączyć z logarytmowaniem.

60) Które stwierdzenia o **cechach numerycznych i jakościowych** są prawdziwe?

- Cechy kateryczne opisują etykiety/klasy (np. **miasto** = "Warszawa"), często wymagają kodowania.
- One-hot encoding stosuje się typowo do cech katerycznych.
- Cechy numeryczne mogą być ciągle (np. temperatura) lub dyskretne (np. liczba sztuk).
- Standaryzacja  $z = (x - \mu) / \sigma$  jest naturalna dla cech numerycznych.
- Mediana i odchylenie standardowe mają sens głównie dla cech numerycznych.

61) Które przykłady wskazują na **niespójne formaty** wymagające ujednolicenia?

- W kolumnie **price** część wartości to "1,99" (przecinek), a część "1.99" (kropka).
- ☐ Wszystkie wartości **price** są float.
- ☐ Wszystkie daty są w ISO 8601 z tą samą strefą czasową.
- ☐ Kolumna logiczna to wyłącznie True/False typu bool.
- Ta sama kolumna dat zawiera "2025-09-26" i "26/09/2025".

62) **Hashing encoding** jest przydatne, gdy:

- ☐ Nigdy nie powoduje kolizji reprezentacji.
- Liczba kategorii jest bardzo duża (wysoka kardynalność).
- Chcemy ograniczyć liczbę kolumn względem one-hot.
- ☐ Kategorie są uporządkowane i jest ich mało.
- ☐ Zawsze wymaga ręcznego przypisania liczb każdej kategorii.

63) Czym jest **standaryzacja z-score**?

- ☐ Zastępowanie braków medianą.
- ☐ Usuwanie wartości odstających powyżej  $3\sigma$ .
- ☐ Przekształcenie do zakresu  $[0, 1]$  przez min-max.
- ☐ Zawsze logarytmowanie danych.
- Przekształcenie  $x \mapsto \frac{x - \mu}{\sigma}$ .



64) **Label encoding** cechy kategorycznej:

- ☐ Jest to to samo co target encoding.
- ☐ Gwarantuje brak wprowadzenia porządku między kategoriami.
- ☐ Tworzy kolumnę dla każdej kategorii z wartościami 0/1.
- ☐ Wymaga z góry znanej liczby kategorii równej 2.
- Przypisuje każdej kategorii liczbę całkowitą  $\{0, 1, 2, \dots\}$ .

65) **One-hot encoding** cechy kategorycznej:

- ☐ Zawsze redukuje liczbę wymiarów w danych.
- Tworzy binarne kolumny wskaźnikowe dla każdej kategorii.
- ☐ Wymaga, by liczba kategorii była równa 2.
- ☐ Jest równoważny label encodingowi.
- Nie wprowadza sztucznego porządku między kategoriami.

66) **Transformacja Yeo-Johnson** względem Box-Coxa:

- Może obsługiwać wartości ujemne i zero.
- ☐ Zawsze daje rozkład idealnie normalny.
- ☐ Nie posiada parametru  $\lambda$ .
- ☐ Jest wyłącznie dla cech kategorycznych.
- ☐ Wymaga  $x > 0$  jak Box-Cox.

67) Jak zweryfikować, że **deduplikacja** nie usunęła poprawnych, unikalnych rekordów?

- ☐ Wyeliminować logowanie operacji, by przyspieszyć pipeline.
- Próbkiwanie usuniętych wierszy i ręczna weryfikacja na podstawie reguł biznesowych.
- Porównanie liczby unikalnych kluczy i rozkładów przed/po deduplikacji.
- ☐ Założyć, że **drop\_duplicates** jest zawsze bezbłędny.
- ☐ Testować na zbiorze testowym i korygować reguły w oparciu o jego wyniki końcowe.

68) Kiedy **label encoding** może być *niebezpieczny* dla modeli liniowych?

- Gdy kategorie są nominalne (bez porządku) — liczby wprowadzają sztuczną rangę.
- Gdy model interpretuje różnice numeryczne jako odległości między kategoriami.
- ☐ Gdy cecha ma tylko dwie klasy binarne.
- ☐ Gdy używamy wyłącznie drzew decyzyjnych.
- ☐ Gdy kategorie są uporządkowane (np.  $S < M < L$ ).

69) Które praktyki pomagają **unikać wycieku** przy tworzeniu cech?

- ☐ Łączenie **train** i **test** do wspólnego liczenia statystyk.
- Używanie **Pipeline**/walidacji krzyżowej do generowania cech z danych treningowych.
- Obliczanie agregatów wyłącznie na **train** i stosowanie tych samych reguł na **valid/test**.
- ☐ Dostrajanie cech w oparciu o metryki na zbiorze testowym.
- ☐ Budowanie cech w oparciu o informacje przyszłe (**t+1**) w zadaniu prognozy **t**.

70) Kiedy użyć **usuwania duplikatów po kluczu** (np. **id**) zamiast pełnego porównania wszystkich kolumn?

- Gdy **id** jest jednoznacznym identyfikatorem rekordu w całym systemie.
- ☐ Gdy chcemy wykrywać jedynie rekordy różniące się wartościami w co najmniej jednej kolumnie.
- ☐ Gdy brak jakiegokolwiek stabilnego identyfikatora.
- ☐ Gdy **id** bywa recyklingowane między różnymi klientami.
- Gdy duplikacja nastąpiła przez wielokrotny import tych samych rekordów pod tym samym kluczem.

71) Które stwierdzenia dotyczące zastąpienie skrajnie dużych (lub małych) wartości ustalonym progiem są poprawne?

- Zastępuje wartości skrajne progami, np. 1. i 99. percentyl.
- ☐ Jest równoważna usuwaniu wierszy z outlierami.
- ☐ Zawsze poprawia każdy model bez wyjątku.
- Powinna być dostrajana na **train** i zastosowana identycznie na **valid/test**.
- ☐ Można ustawić progi po obejrzeniu wyników na **test**.

72) **PowerTransformer** (Box–Cox / Yeo–Johnson) — które zdania są prawdziwe?

- Yeo–Johnson obsługuje wartości  $\leq 0$ , Box–Cox wymaga  $x > 0$ .
- ☐ PowerTransformer zawsze skaluje do  $[0, 1]$ .
- ☐ Box–Cox działa najlepiej dla danych katerycznych.
- ☐ Yeo–Johnson to to samo co  $\log(x + 1)$ .
- Obie metody dopasowują parametr  $\lambda$  w celu „unormalnienia” rozkładu.

73) Które przykłady wskazują na **niespójne jednostki lub skale** wymagające normalizacji/skalowania?

- Kolumny **revenue** (w milionach) i **count** (w sztukach) są używane razem w modelu odległościowym.
- ☐ Wszystkie pomiary są w **cm**.
- ☐ Kolumna kateryczna **color** ma etykiety **"red", "green", "blue"**.
- ☐ Wszystkie cechy są już w  $[0, 1]$  po min–max.
- Część rekordów ma **height** w **cm**, a część w **inch**.

74) Które podejścia są poprawnymi *strategiami uzupełniania braków* (**NaN**) w danych liczbowych?

- Imputacja średnią/medianą z **train**, np. **median**.
- ☐ Mnożenie **NaN** przez 0, aby „zniknęły”.
- ☐ Losowe zastąpienie **NaN** liczbą 999 bez względu na skalę.
- ☐ Zawsze usuwanie całych kolumn z dowolną liczbą braków.
- Imputacja metodą KNN (np. *K-Nearest Neighbors*).

75) W której sytuacji konieczne jest **usuwanie duplikatów**?

- Występują duplikaty po kluczu **id** z identycznym **timestamp**.
- ☐ Zliczenia w tabeli przestawnej powielają wartości agregatów.
- ☐ Występują powtarzające się kategorie w słowniku mapującym.
- Te same rekordy występują wielokrotnie wskutek łączenia tabel.
- ☐ Rekordy różnią się wartościami, ale mają tę samą klasę docelową.

76) **Hashing encoding** ma następujące cechy:

- ☐ Jest identyczny z one–hot encodingiem.
- Nie wymaga słownika wszystkich kategorii.
- ☐ Wymaga pełnej listy kategorii przed dopasowaniem.
- Rzutuje kategorie do przestrzeni o stałym wymiarze, co może powodować kolizje.
- ☐ Gwarantuje brak kolizji dla dowolnej liczby kategorii.

77) Które stwierdzenia o **mechanizmach braków** są poprawne?

- ☐ MCAR i MAR to to samo, różnią się tylko nazwą.
- ☐ MNAR oznacza, że braków nie ma wcale.
- ☐ MNAR: brak zależy wyłącznie od etykiety **y**, więc można go pominąć.
- MCAR: brak losowy, niezależny od obserwowanych i nieobserwowanych zmiennych.
- MAR: brak zależy od obserwowanych zmiennych (możliwy do modelowania).

78) Po **wdrożeniu** modelu, które praktyki związane z danymi są kluczowe, aby utrzymać jakość rozwiązania?

- ☐ Trwałe łączenie zbiorów **train** i **test** w jeden plik, żeby „uśrednić” rozkłady.
- ☐ Zamrożenie wersji danych na stałe i zakaz aktualizacji.
- Przyrostowe etykietowanie nowych przykładów i okresowe odświeżanie zbioru **train** do retrenowania.
- Monitorowanie dystrybucji cech i metryk oraz zbieranie sprzężenia zwrotnego.
- ☐ Wyłączanie logowania danych wejściowych, by zmniejszyć koszty monitoringu.

79) Które **statystyki odporne** (robust) warto rozważyć przy obecności outlierów?

- Mediana i median absolute deviation (MAD).
- ☐ Średnia arytmetyczna zawsze lepsza od mediany przy długim ogonie.
- ☐ Odchylenie standardowe jest niewrażliwe na outliery.
- Percentyle (np. 5., 95.) zamiast min/max.
- ☐ Kwantyle są bezużyteczne w ocenie rozkładów.

80) **Dyskretyzacja** (binning) cechy numerycznej polega na:

- ☐ Zastąpieniu wartości średnią z kolumny.
- ☐ Losowym permutowaniu wartości w celu anonimizacji.
- Podziale zakresu wartości na przedziały i przypisaniu obserwacji do binów.
- ☐ Zawsze na skalowaniu do  $[0, 1]$ .
- ☐ One-hot encodingu unikalnych wartości liczbowych.

81) Które metody **wykrywania wartości odstających** są poprawne dla danych jednowymiarowych?

- Reguła IQR: obserwacje poza  $[Q_1 - 1.5 \cdot IQR, Q_3 + 1.5 \cdot IQR]$ .
- ☐ Losowe wskazanie 5
- z-score  $|z| > 3$  przy (w przybliżeniu) rozkładzie normalnym.
- ☐ Sprawdzenie, czy wartość jest równa średniej arytmetycznej.
- ☐ Uznać wartości o największej częstości za odstające.

82) (Python/sklearn) Które techniki służą do **detekcji anomalii** w danych wielowymiarowych?

- ☐ LabelEncoder na zmiennej celu.
- ☐ One-Hot Encoding.
- Isolation Forest.
- LOF (Local Outlier Factor).
- ☐ StandardScaler.

83) (Python/sklearn) Które praktyki pomagają **uniknąć wycieku** i błędów w preprocessing'u?

- ☐ Dopasuj **StandardScaler** na całym zbiorze, a potem podziel na **train/test**.
- Używaj **Pipeline/ColumnTransformer** do spójnego przetwarzania.
- ☐ Losuj **train/test** po dokonaniu one-hot encodowania na całym zbiorze.
- ☐ Uzupełniaj braki średnią policzoną na połączonym **train+test**.
- Dziel **train/test** przed dopasowaniem skalera/imputera; dopasuj na **train**, zastosuj na **test**.

84) (Python/sklearn) **RobustScaler** w scikit-learn:

- Odejmuje medianę i dzieli przez IQR ( $Q_3 - Q_1$ ).
- ☐ Odejmuje średnią i dzieli przez odchylenie standardowe.
- ☐ Skaluje zawsze do  $[0, 1]$  na podstawie min i max.
- ☐ Stosuje kodowanie one-hot dla cech katagorycznych.
- ☐ Wykonuje logarytmowanie danych automatycznie.

85) (Python/sklearn) Które działania pomagają **unikać wycieku** przy obchodzeniu się z outlierami?

- Dopasowanie progów/winsoryzacji wyłącznie na **train**, potem zastosowanie na **valid/test**.
- ☐ Ustalanie progów na całym zbiorze, a potem podział na **train/test**.
- ☐ Zastąpienie skrajnie dużych (lub małych) wartości ustalonym progiem w oparciu o wartości docelowe y.
- ☐ Dobór progów na podstawie metryk z **test**.
- Stosowanie **Pipeline/ColumnTransformer** do spójnego przetwarzania.

86) (Python/sklearn) Kiedy **RobustScaler** jest preferowany względem **StandardScaler**?

- ☐ Gdy chcemy automatycznie usunąć outliery ze zbioru.
- Gdy rozkłady są ciężkoogonowe i średnia/ $\sigma$  są niestabilne.
- Gdy w danych występują silne outliery i chcemy użyć mediany/IQR.
- ☐ Gdy potrzebujemy dokładnie zakresu  $[0, 1]$ .
- ☐ Gdy wszystkie cechy są już binarne.

87) (Python/pandas) Jak usunąć **dokładne duplikaty** wierszy w `pandas.DataFrame` **df** i zachować pierwsze wystąpienie?

- `df = df.drop_duplicates(keep="first")`
- ☐ `df = df.drop_duplicates(keep=False)`
- `df = df[~df.duplicated(keep="first")]`
- ☐ `df = df.unique()`
- ☐ `df = df.dropna()`

88) (Python/pandas) Jak **interpolować** brakujące wartości w szeregu czasowym `df["temp"]` (`pandas.DataFrame`)?

- ☐ `df["temp"] = df["temp"].binarize()`
- `df["temp"] = df["temp"].interpolate(method="time")`
- ☐ `df["temp"] = df["temp"].mean(method="time")`
- `df["temp"] = df["temp"].interpolate(method="linear")`
- ☐ `df["temp"] = df["temp"].interpolate(method="drop")`

89) Które przykłady *zadań ML* są typowe?

- Klasteryzacja i redukcja wymiarowości.
- ☐ Sortowanie tablicy przez ręcznie zakodowany algorytm bez danych treningowych.
- Klasyfikacja i regresja.
- ☐ Renderowanie 3D scen w czasie rzeczywistym bez żadnej optymalizacji na danych.
- Rekomendacje i detekcja anomalii.

90) W jakich scenariuszach *semi-supervised* jest szczególnie uzasadnione?

- Gdy etykietowanie jest drogie, a nieoznaczonych danych jest dużo.
- ☐ Gdy zadanie wymaga nagród sekwencyjnych za działania w środowisku.
- ☐ Gdy mamy pełne, dokładne etykiety dla wszystkich przykładów i brak ograniczeń budżetowych.
- ☐ Gdy chcemy wyłącznie grupować bez jakiegokolwiek sygnału etykiet.
- ☐ Gdy trenowanie odbywa się wyłącznie na urządzeniach klientów bez wymiany parametrów.

91) Które stwierdzenia o *funkcjach straty i treningu* są trafne?

- Klasyfikacja często używa entropii krzyżowej (log-loss).
- Optymalizacja dokładności bezpośrednio gradientowo jest trudna, więc używa się np. cross-entropy.
- ☐ Klasyfikacja nie potrzebuje regularizacji — wystarczy zwiększyć liczbę epok.
- ☐ W regresji zawsze lepsza jest MSE niż Huber/MAE, bo silniej karze duże błędy.
- ☐ AUC-ROC jest łatwo różniczkowalną funkcją i powszechnie używa się jej bezpośrednio jako straty w SGD.

92) Czy ML różni się od *programowania tradycyjnego*?

- W tradycyjnym podejściu: *reguły + dane* → *wyniki*; w ML: *dane + wyniki (etykiety)* → *reguły/model*.
- ☐ Programowanie tradycyjne nie korzysta z danych wejściowych.
- ☐ ML polega wyłącznie na zwiększaniu częstotliwości CPU.
- ML uczy parametry z danych i może adaptować się po wdrożeniu; kod regułowy działa według ręcznie zapisanej logiki.
- ☐ ML zawsze działa bez funkcji celu/straty.

93) Które stwierdzenia o *reprodukowalności* eksperymentów ML są poprawne?

- ☐ Wystarczy zapisać tylko wynik Accuracy — reszta nie ma znaczenia.
- Logowanie wersji danych, kodu, metryk i konfiguracji zwiększa powtarzalność.
- Ustalanie ziaren losowości (`random_state/seed`) zmniejsza wariancję wyników między uruchomieniami.
- ☐ Równoległość nie ma wpływu na deterministyczność algorytmów.
- ☐ Zmiana wersji bibliotek nie wpływa na wyniki — można ją ignorować.

94) Krzywe uczenia (learning curves) dla *underfittingu* zwykle pokazują:

- Wysoki błąd treningowy i walidacyjny, małą lukę między nimi nawet przy dużej liczbie próbek.
- ☐ Niski błąd treningowy, wysoki walidacyjny i duża luka między nimi.
- ☐ Oscylacje błędu wynikające wyłącznie z losowania minibatchy bez trendu.
- ☐ Idealnie zerowy błąd na obu zbiorach od pierwszej epoki.
- ☐ Walidacja zawsze poniżej treningu o stałe 0.5.

95) *Uczenie przez transfer (Transfer Learning)* polega na:

- ☐ Nadpisaniu wszystkich warstw stałymi zerami, by „wyczyścić” wiedzę.
- ☐ Zawsze trenowaniu modelu od zera na zadaniu docelowym bez użycia pretrenowanych wag.
- Wykorzystaniu wiedzy z zadania/źródła (modelu wytrenowanego na domenie źródłowej) do poprawy wyników w zadaniu docelowym (np. fine-tuning).
- ☐ Uczeniu polityki w środowisku bez wykorzystania wcześniejszych reprezentacji.
- ☐ Wyłącznie kopiowaniu etykiet ze zbioru źródłowego na docelowy.

96) Która definicja najlepiej opisuje *uczenie maszynowe (ML)*?

- ☐ Synonim programowania obiektowego z dziedziczeniem wielokrotnym.
- ☐ Standard zapisu grafiki wektorowej do prezentacji wyników.
- ☒ Dziedzina, w której systemy poprawiają swoje działanie na podstawie doświadczenia, mierzonego według miary jakości.
- ☐ Zbiór ręcznie zapisanych reguł, które nigdy nie zmieniają się pod wpływem danych.
- ☐ Proces zwiększania taktowania CPU w celu przyspieszenia programu.

97) Które twierdzenia o *Active Learning* są prawdziwe?

- ☐ Active learning nie wymaga żadnych technik etykietowania — etykiety generują się same.
- ☐ Active learning gwarantuje osiągnięcie 100% dokładności przy dowolnie małej liczbie etykiet.
- ☐ Active learning to synonim Federated Learning — chodzi o federację klientów.
- ☐ Najlepszą strategią jest zawsze wybierać przykłady, dla których model jest najbardziej pewny.
- ☐ Wybór próbek nie wpływa na uprzedzenia — nie trzeba dbać o reprezentatywność.

98) *Uczenie samonadzorowane (Self-Supervised Learning)* polega na:

- ☒ Tworzeniu zadań pretekstu z samych danych i uczeniu reprezentacji bez ręcznych etykiet.
- ☐ Wyłączenie uczeniu polityki w środowisku z nagrodą.
- ☐ Wymogu federacyjnego szkolenia na wielu klientach.
- ☐ Ręcznym definiowaniu tysięcy reguł klasyfikacji dla każdego przypadku.
- ☐ Zawsze pełnym nadzorze z dokładnymi etykietami.

99) Dopasuj rodzaj uczenia do *charakteru etykiet*.

- ☐ Self-Supervised → wymaga ręcznych etykiet dla wszystkich próbek.
- ☒ Semi-Supervised → mieszanka etykietowanych i nieetykietowanych przykładów.
- ☒ Weakly Supervised → etykiety nieprecyzyjne/szumne/słabe.
- ☒ Supervised → pełne, wiarygodne etykiety dla próbek.
- ☐ Reinforcement Learning → etykiety klas dla każdego kroku, brak nagród.

100) Dopasuj *metryki jakości* do typu zadania.

- ☐ RMSE to podstawowa metryka klasyfikacji binarnej.
- ☒ Dokładność/Accuracy, F1, AUC-ROC → klasyfikacja.
- ☐ Log-loss (cross-entropy) → wyłącznie regresja liniowa.
- ☒ MAE, MSE/RMSE,  $R^2$  → regresja.
- ☐ AUC-PR jest standardową metryką regresji bez progowania.

101) W Reinforcement Learning (RL), w porównaniu do supervised, różnica w sygnale nadzoru polega na tym, że:

- ☐ W supervised brak jest jakiegokolwiek funkcji celu/straty.
- ☐ W RL zawsze znamy prawidłową akcję w każdym stanie.
- ☐ W supervised decyzje zależą wyłącznie od nagród i polityki.
- ☐ W RL nie występuje problem eksploracja–eksploatacja.
- ☒ Zamiast etykiet docelowych dla każdej próbki, agent dostaje nagrody zależne od sekwencji akcji i stanu środowiska.

102) Które stwierdzenia o *stratyfikacji* w k-fold są prawdziwe?

- ☐ Stratyfikacja jest zabroniona w problemach z niebalansowanymi klasami.
- ☐ Stratyfikacja wymaga, by ten sam przykład znalazł się w wielu foldach naraz.
- ☐ W regresji stratyfikacja zawsze rozdziela obserwacje według identycznych wartości ciągłych bez binowania.
- ☐ Stratyfikacja to wyłącznie technika Reinforcement Learning do doboru akcji.
- ☐ Stratyfikacja polega na mieszaniu etykiet między foldami.

103) Co jest *celem* typowego procesu ML z punktu widzenia uogólniania?

- ☒ Kontrola przeuczenia m.in. przez regularizację i walidację.
- ☐ Maksymalizacja rozmiaru pliku modelu w MB.
- ☐ Losowe ustawianie wag, by uniknąć biasu.
- ☒ Minimalizacja błędu generalizacji poprzez uczenie z danych i ocenę na danych niewidzianych.
- ☐ Doskonałe dopasowanie do zbioru treningowego kosztem reszty.

104) Jakie *rodzaje błędów* i ryzyk etapu przygotowania danych są typowe?

- Data leakage: dopasowanie transformacji na całych danych przed podziałem.
- Brak wersjonowania danych i pipeline’u transformacji.
- Niespójne enkodowanie kategorii między train a test.
- Target leakage: cechy zależne od etykiety lub przyszłości chwili predykcji.
- Nadmiarowy cleaning usuwający informację.

105) *Uczenie federacyjne (Federated Learning)* oznacza, że:

- ☐ Zawsze gwarantuje pełną anonimowość bez dodatkowych mechanizmów.
- Model jest trenowany współbieżnie na wielu klientach/urządzeniach; do serwera przekazywane są aktualizacje/gradientsy lub wagi, nie surowe dane.
- Celem jest m.in. poprawa prywatności i zgodności regulacyjnej przez lokalne przetwarzanie danych.
- ☐ Federacja jest tożsama z Reinforcement Learning — wymaga nagród i polityk.
- ☐ Serwer zbiera wszystkie surowe dane klientów do jednej bazy.

106) Które zdania o *Federated Learning (FL)* są prawdziwe?

- ☐ FL zawsze gwarantuje pełną prywatność bez dodatkowych technik.
- ☐ FL nie wymaga żadnej komunikacji między serwerem a klientami.
- ☐ FL to to samo co Reinforcement Learning.
- ☐ W FL surowe dane wszystkich klientów są scalane na serwerze.
- ☐ FL jest niemożliwy, gdy dane klientów mają różne rozkłady.

107) Jak rozpoznać *nadmierną wariancję* modelu (high variance)?

- ☐ Identyczne metryki na wszystkich foldach niezależnie od losowania.
- ☐ Brak wrażliwości na zmiany hiperparametrów i losowania.
- ☐ Słabe wyniki na treningu i na walidacji.
- Duża rozbieżność między wynikiem na treningu (bardzo dobry) a walidacją/testach (wyraźnie gorszy).
- ☐ Niższa wariancja metryk przy mniejszym zbiorze treningowym.

108) Czym różnią się *selekcja cech* od *redukcji wymiarowości*?

- Selekcja wybiera podzbiór oryginalnych cech, a redukcja tworzy nowe, transformowane cechy.
- ☐ Selekcja wymaga etykiet, a redukcja — zawsze działa bez danych.
- ☐ To identyczne pojęcia; różni je wyłącznie nazewnictwo.
- ☐ Selekcja zawsze zwiększa liczbę cech, redukcja — nigdy.
- ☐ Redukcja działa tylko dla danych binarnych; selekcja — tylko dla ciągłych.

109) Czym różnią się *parametry* od *hiperparametrów* w ML?

- ☐ Hiperparametry dotyczą wyłącznie sprzętu (GPU/CPU), parametry — wyłącznie danych.
- ☐ Parametry i hiperparametry to synonimy — oba są uczone gradientem.
- ☐ Parametry ustala człowiek ręcznie, hiperparametry wyznacza regresja liniowa.
- ☐ Parametry są zawsze liczbami całkowitymi, hiperparametry — rzeczywistymi.
- Parametry są uczone z danych, a hiperparametry ustala się przed/w trakcie treningu przez walidację.

110) Które stwierdzenia o *Transfer Learningu* są prawdziwe?

- Adaptacja domenowa stara się zredukować rozbieżność rozkładów między źródłem a celem.
- Ryzykiem jest „negative transfer”, gdy domeny są zbyt różne.
- Można łączyć z Active Learning, by efektywnie dobierać próbki do etykietowania.
- Zamrażanie części wag zmniejsza przeuczenie przy małej liczbie danych.
- Może znacząco redukować zapotrzebowanie na etykiety w zadaniu docelowym.

111) *Uczenie częściowo nadzorowane (Semi-Supervised Learning)* zakłada, że:

- Można stosować techniki konsystencji/entropii, pseudo-etykietowanie i regularizację na danych bez etykiet.
- Dysponujemy mieszanką przykładów oznaczonych i nieoznaczonych, a model wykorzystuje obie pule do poprawy jakości.
- ☐ Uczenie to wyłącznie eksploracja/eksploatacja z nagrodą.
- ☐ Wymaga federacyjnej orkiestracji klient-serwer.
- ☐ Wszystkie przykłady są perfekcyjnie oznaczone — danych bez etykiet nie używa się.

112) *Uczenie nienadzorowane (Unsupervised Learning)* najlepiej opisuje:

- ☐ Wymóg posiadania funkcji nagrody i polityki działania.
- ☐ Wyłącznie kompresję plików bez odniesienia do danych.
- ☒ Odkrywanie struktury danych bez etykiet.
- ☐ Uczenie z dokładnymi etykietami dla każdego przykładu.
- ☐ Modyfikację wag modelu wyłącznie ręcznie przez programistę.

113) Które działania są typowymi *sposobami ograniczania overfittingu*?

- ☒ Uproszczenie modelu lub redukcja liczby parametrów.
- ☒ Więcej danych/augmentacja danych.
- ☐ Usunięcie zbioru walidacyjnego.
- ☐ Zwiększenie liczby epok bez jakiegokolwiek regularizacji lub monitoringu.
- ☒ Regularizacja (np.  $L_2/L_1$ , dropout, wczesne zatrzymanie).

114) Które stwierdzenia o *weakly vs. semi-supervised* są poprawne?

- ☐ Weakly supervised wymaga, aby wszystkie próbki były perfekcyjnie oznaczone.
- ☒ Semi-supervised dotyczy ilości etykiet: część próbek ma etykiety, część nie.
- ☐ Oba pojęcia oznaczają dokładnie to samo w każdym kontekście.
- ☒ Weakly supervised koncentruje się na jakości/sile etykiety (szum, nieprecyzja, etykiety pośrednie).
- ☐ Semi-supervised zakłada brak danych bez etykiet.

115) Które techniki należą do *Transfer Learning*?

- ☒ Feature extraction z zamrożonym szkieletem i nową warstwą klasyfikacyjną.
- ☒ Adaptacja domenowa.
- ☐ Wyłącznie regularyzacja L2 bez przenoszenia parametrów/cech.
- ☒ Fine-tuning części/całości sieci pretrenowanej.
- ☐ Losowe inicjalizowanie wag za każdym razem bez wykorzystania wiedzy źródłowej.

116) W którym paradygmacie *nie* korzysta się z ręcznie nadanych etykiet podczas pre-treningu, ale wykorzystuje się je często przy fine-tuningu?

- ☐ Reinforcement learning z nagrodą natychmiastową.
- ☐ Uczenie nadzorowane.
- ☒ Uczenie samonadzorowane.
- ☐ Uczenie nienadzorowane z k-means.
- ☐ Federated learning z centralizacją danych.

117) Które paradygmaty *uczenia* poprawnie przypisano do ML?

- ☐ Uczenie „kompilacyjne”, w którym poprawa wyniku wyłącznie z optymalizacji kodu asemblera.
- ☐ Uczenie deterministyczne, w którym losowość jest prawnie zakazana.
- ☐ Uczenie „bez danych”, w którym model nie potrzebuje żadnych przykładów.
- ☒ Uczenie nadzorowane, nienadzorowane i samonadzorowane.
- ☒ Uczenie ze wzmocnieniem (RL) z funkcją nagrody.

118) Co to jest *przekleństwo wymiarowości*?

- ☒ Odległości między punktami ulegają „spłaszczeniu”, co szkodzi metodom opartym na sąsiadach/metrykach.
- ☒ W wysokich wymiarach dane stają się rzadkie; potrzeba wykładniczo więcej próbek do stabilnej estymacji.
- ☐ Można go pominąć, ustawiając `random_state`.
- ☐ Wysoki wymiar zawsze pomaga generalizacji, bo model ma więcej informacji.
- ☐ Dotyczy wyłącznie modeli liniowych; metody nieliniowe są odporne.

119) (Python/sklearn) Dla których algorytmów *skalowanie cech* jest zwykle kluczowe?

- ☒ `LogisticRegression`
- ☐ `DecisionTreeClassifier`
- ☐ `RandomForestClassifier`
- ☒ `KNeighborsClassifier`
- ☒ `SVC / LinearSVC`

120) (Python/sklearn) Co oznacza hiperparametr  $\alpha$  w Ridge i Lasso?

- Siłę regularyzacji: większe  $\alpha$  silniej karze duże wagi.
- ☐ Współczynnik uczenia `learning_rate` jak w boostingu.
- ☐ Stopień wielomianu w `PolynomialFeatures`.
- ☐ Liczbę epok gradientu — większe  $\alpha$  to dłuższy trening.
- ☐ Liczbę drzew w `RandomForestRegressor`.

121) (Python/sklearn) Dopasuj wariant *Naive Bayes* do typu danych.

- `GaussianNB` → cechy ciągłe o (przybliżonym) rozkładzie normalnym.
- ☐ `GaussianNB` → wyłącznie dane binarne 0/1.
- ☐ `CategoricalNB` → surowe piksele RGB bez kategoryzacji jako wartości ciągłe.
- `MultinomialNB` → zliczenia/słownictwo.
- `BernoulliNB` → cechy binarne (0/1).

122) (Python) Które pakiety zaliczysz do *podstawowego stosu ML/Data* w Pythonie?

- pandas — ramki danych, ETL i przygotowanie cech.
- ☐ requests — wykonywanie zapytań HTTP jako główny silnik uczenia.
- scikit-learn — modele klasyczne, Pipeline, CV, GridSearch.
- matplotlib — wykresy i wizualizacja.
- NumPy — obliczenia tablicowe i wektoryzacja.

123) (Python/sklearn) Które twierdzenia o `RandomForestClassifier` są prawdziwe?

- ☐ To metoda *boostingu* sekwencyjnego.
- Jest to *bagging*: bootstrap próbek + losowe podzbiory cech.
- `feature_importances_` szacuje ważności cech.
- `oob_score=True` umożliwia ocenę out-of-bag.
- ☐ Wymaga standaryzacji cech, inaczej nie działa poprawnie.

124) (Python/sklearn) Które stwierdzenia o *strategiach wieloklasowych* są poprawne?

- `LinearSVC` używa One-vs-Rest (OvR).
- ☐ `LogisticRegression` nie obsługuje klasy wieloklasowej.
- `SVC` domyślnie stosuje One-vs-One (OvO).
- ☐ `LinearSVC` udostępnia `predict_proba` dla OvR.
- `LogisticRegression` wspiera `multi_class="multinomial"` (softmax) dla solverów `lbfgs/newton-cg/saga`.

125) (Python/sklearn) Metryki dla regresji: które przypisania są poprawne?

- ☐ AUC-ROC/AUC-PR → standardowe metryki regresji bez progów.
- ☐ Accuracy → podstawowa metryka regresji ciągłej.
- ☐ Log-loss (cross-entropy) → typowa metryka regresji.
- `mean_pinball_loss` → regresja kwantylowa.
- `MAE/MSE/RMSE/R2` → regresja.

126) (Python/sklearn) Po co stosować *zagnieżdżoną walidację krzyżową* przy strojeniu hiperparametrów?

- ☐ Aby losowo mieszać etykiety i sprawdzić, czy model jest „kreatywny”.
- ☐ Aby móc trenować dwa różne modele jednocześnie na tym samym foldzie.
- Aby rozdzielić wybór hiperparametrów (pętla wewnętrzna CV) od *bezzstronnej* oceny końcowej (pętla zewnętrzna CV) i ograniczyć optymizm estymatora jakości.
- ☐ Aby pominąć potrzebę posiadania zbioru testowego i walidacyjnego.
- ☐ Aby móc używać metryk niedostępnych w zwykłej walidacji (np. Accuracy).

127) (Python/sklearn) Które algorytmy to podstawowe *regresory* w `scikit-learn`?

- `RandomForestRegressor`
- ☐ `LogisticRegression`
- `LinearRegression`
- Ridge / Lasso / ElasticNet
- ☐ `MultinomialNB`



128) (Python/sklearn) `KNeighborsRegressor`: kiedy ma sens i o co zadbać?

- Wymaga skalowania cech; podatny na „curse of dimensionality”.
- Nie wpływa na niego metryka; zawsze używa kosinusowej.
- Zawsze przewyższa boosting w dużych danych.
- Ma etap uczenia z dużą liczbą epok jak w sieciach neuronowych.
- Działa dobrze przy lokalnie gładkich zależnościach — dobór `n_neighbors/metric/weights` jest kluczowy.

129) (Python/sklearn) Modelami *klasyfikacji* w `scikit-learn` są:

- `SVC`
- `LinearRegression`
- `LogisticRegression`
- `RandomForestClassifier`
- `Lasso`

130) (Python/sklearn) Które z poniższych to *liniowe* klasyfikatory w `scikit-learn`?

- `LinearSVC`
- `LogisticRegression`
- `KNeighborsClassifier`
- `Perceptron`
- `DecisionTreeClassifier`

131) (Python) Dopasuj bibliotekę do *zadania/zastosowania*.

- Optuna → automatyczna optymalizacja hiperparametrów z integracją `sklearn` i callbackami.
- imbalanced-learn (`imblearn`) → techniki radzenia sobie z niezbalansowanymi klasami i integracja z `sklearn.Pipeline`.
- MLflow → śledzenie eksperymentów, artefaktów i rejestr modeli.
- skorch → adapter ujednolicejący modele PyTorch do interfejsu `sklearn`.
- XGBoost/LightGBM/CatBoost → gradient boosting z wrapperami API kompatybilnymi z `sklearn` i opcjonalnym wsparciem GPU.

132) (Python/sklearn) Jak poprawnie zbudować regresję wielomianową?

- Dodać losowy szum do cech zamiast wielomianu.
- Zawsze użyć `RandomForestRegressor` — to „wielomian z natury”.
- Podnieść wektor  $y$  do potęgi i wytrenować `LinearRegression`.
- Zastąpić `PolynomialFeatures` `LabelEncoderem`.
- Użyć `Pipeline` z `PolynomialFeatures` (opcjonalnie + `scaler`) → `LinearRegression/Ridge`.

133) Które elementy są charakterystyczne dla *sieci semantycznych*?

- Sieci semantyczne nie pozwalają na relacje hierarchiczne.
- Mają obowiązkowo dokładnie jeden rodzaj krawędzi.
- Węzły to zawsze liczby rzeczywiste, krawędzie — operatory arytmetyczne.
- Są równoważne drzewom decyzyjnym do klasyfikacji.
- Węzły reprezentują pojęcia/instancje, krawędzie — typy relacji (np. `is-a`, `part-of`); możliwa propagacja dziedziczenia.

134) Które stwierdzenia o *wiedzy heurystycznej* są prawdziwe?

- Może działać jako priorytetyzacja akcji/gałęzi w planowaniu.
- Ma charakter „zdroworozsądkowych wskazówek” lub funkcji oceny przybliżającej, upraszczających przeszukiwanie.
- Bywa pozyskiwana od ekspertów lub uczona z danych.
- Może być niedokładna, ale przyspiesza znajdowanie dobrych rozwiązań.
- Nie gwarantuje optymalności, chyba że spełnia warunki (np. *admissible*, *consistent* w  $A^*$ ).

135) Zaznacz zdania, które *nie należą* do opisu grafów semantycznych.

- Grafy semantyczne nie wspierają typów krawędzi ani hierarchii.
- Dziedziczenie w grafach jest niemożliwe.
- Relacje w grafie muszą być komutatywne i bezkierunkowe.
- Każda krawędź musi wskazywać na węzeł liczbowy „bias”.
- Węzły są wyłącznie liczbami zmiennoprzecinkowymi bez etykiet.

136) Klauzule Horna i programowanie w logice (Datalog/Prolog): wskaż prawdziwe.

- ☐ Prolog nie używa unifikacji w dopasowaniu wzorców.
- Klauzula Horna ma co najwyżej jedną literę dodatnią.
- ☐ Klauzule Horna nie mogą reprezentować reguł IF–THEN.
- ☐ Datalog dopuszcza funkcje o nieskończonej głębokości, gwarantując terminację.
- Wnioskowanie może używać SLD-resolution z unifikacją.

137) Czym charakteryzuje się *wiedza deklaratywna* w systemach SI?

- ☐ Zawsze musi mieć formę sieci Bayesa — inne formy są niepoprawne.
- ☐ To wyłącznie wagi sieci neuronowej bez warstwy semantycznej.
- ☐ To procedury i kroki „jak coś zrobić” zakodowane w algorytmie.
- ☐ Nie może być zapisana w logice — tylko w pseudokodzie.
- Opisuje *fakty i relacje* o świecie, niezależnie od algorytmu użycia.

138) Resource Description Framework/Triple store: które pary są poprawnymi trzema składowymi trójki?

- ☐ *Wartość–Funkcja straty–Gradient*.
- *Podmiot–Predykat–Dopełnienie* (Subject–Predicate–Object).
- ☐ *Źródło–Waga–Bias* jako jedyny dopuszczalny format.
- Relacje kodowane są jako predykaty (IRI), obiekty mogą być IRI lub literałami.
- ☐ *Klasa–Cecha–Etykieta prawdy* (*True/False*) wyłącznie.

139) Które twierdzenia o *wnioskowaniu do przodu* vs *do tyłu* są prawdziwe?

- ☐ Do przodu zawsze jest bardziej efektywne od do tyłu.
- ☐ Do tyłu wymaga znajomości wszystkich faktów początkowych w bazie.
- Do tyłu: od celu szukamy reguł, które mogą go udowodnić, redukując do podcelów.
- Do przodu: od faktów uruchamiamy reguły i dodajemy nowe fakty, aż do nasycenia lub osiągnięcia celu.
- ☐ Żadna z tych technik nie stosuje unifikacji.

140) Czym różnią się *grafy semantyczne* od *ram* (frames)?

- ☐ Grafy wymagają języka Web Ontology Language, ramy — tylko logiki zdań.
- Grafy semantyczne modelują pojęcia jako węzły i relacje jako krawędzie; ramy to ustrukturyzowane „rekordy” (klasy) z atrybutami (slotami) i domyślnymi wartościami.
- ☐ Grafy są wyłącznie acykliczne, ramy muszą zawierać cykle.
- ☐ Ramy są losowymi lasami drzew bez atrybutów; grafy to zbiory funkcji slotów.
- ☐ Ramy nie obsługują dziedziczenia, grafy — zawsze.

141) Które charakterystyki reguł *Horn* są poprawne?

- Specjalny przypadek: fakty (brak przesłanek) i cele (brak konkluzji).
- Są podstawą Datalogu i wielu systemów regułowych.
- Dobrze wspierają wnioskowanie do przodu i do tyłu.
- Efektywne wnioskowanie (względem ogólnej logiki pierwszego rzędu) w wielu zadaniach praktycznych.
- Postać:  $A \leftarrow B_1 \wedge \dots \wedge B_n$  (konkluzja i koniunkcja przesłanek).

142) Ramy (frames) Minsky’ego: które stwierdzenia są *prawdziwe*?

- Umożliwiają procedury przy slotach (tzw. *demons/triggers*) wywoływane przy odczycie/zapisie.
- Łączą wiedzę deklaratywną (struktura) z proceduralną (procedury slotów).
- Są wygodne do modelowania stereotypowych sytuacji (schematy/„scripts”).
- Pozwalają na dziedziczenie domyślnych wartości po hierarchii ram.
- Reprezentują pojęcia jako struktury z atrybutami (slotami) i wartościami (fillerami).

143) Co *dodaje* logika predykatów względem logiki zdań?

- ☐ Reprezentację probabilistyczną Bayesowską.
- Kwantyfikatory ( $\forall, \exists$ ), zmienne, funkcje i relacje pozwalające wyrażać własności i związki obiektów.
- ☐ Losowe ważenie formuł zamiast semantyki modelowej.
- ☐ Wyłącznie rachunek lambda bez relacji.

☐ Mechanizm dziedziczenia jak w grafach semantycznych.

144) (Python) Które przykłady są poprawnymi *regułami Horna*?

☐ `parent(X,Y) ; sibling(X,Y) :- cousin(X,Y).`

■ `grandparent(X,Z) :- parent(X,Y), parent(Y,Z).`

☐ `:- parent(X,Y).`

■ `ancestor(X,Z) :- parent(X,Z).`

☐ `A(X) :- not B(X).`

145) (Python) Które sytuacje powodują *konflikt substytucji* podczas unifikacji?

☐ Wzorzec i fakt mają tę samą arność oraz zgodne nazwy predykatów.

■ Substytucja zawiera `?x: 'anna'`, a fakt wymaga `?x == 'eva'`.

■ Nazwy predykatów są różne mimo tej samej arności (np. `p(...)` kontra `q(...)`).

☐ Substytucja jest pusta, a wzorzec ma tylko zmienne.

☐ Fakt ma stałe, a wzorzec — zmienne na tych pozycjach.

146) (Python) Kiedy dopasowanie ciała reguły `((A_1,...,A_k))` do faktów *musi* się nie powieść?

☐ Jeśli w ciele występują co najmniej dwie zmienne.

☐ Jeśli istnieje choć jeden fakt zgodny z `(A_1)`.

■ Jeśli dla któregoś `(A_i)` brak jest jakiegokolwiek zgodnego faktu pod już zgromadzonymi wiązaniami.

☐ Jeśli predykaty w ciele są zapisane małymi literami.

■ Jeśli arność któregoś `(A_i)` różni się od arności faktów o tej nazwie predykatu.

147) (Python) Kiedy unifikacja `parent('?'x','ola')` ze zmienną `'?x'` z `parent('eva','ola')` powiedzie się i jaką da substytucję?

■ Powiedzie się z substytucją: `?x -> 'eva'`.

☐ Powiedzie się z substytucją: `?x -> 'ola'`.

☐ Nie powiedzie się, bo fakt jest ugruntowany.

☐ Powiedzie się tylko przy odwróconej kolejności argumentów.

☐ Nie powiedzie się, bo wzorzec zawiera zmienną.

148) (Python) Wybierz poprawne interpretacje *zapytania wzorcowego* po nasyceniu.

■ Po rekurencyjnym wnioskowaniu wyniki mogą obejmować zależności tranzytywne.

■ Pusty wynik oznacza brak odpowiadających faktów w zmaterializowanej bazie.

■ Wildcard na pozycji argumentu dopasowuje dowolną wartość.

■ Zapytanie zwraca listę krotek argumentów faktów zgodnych z wzorcem.

■ Kolejność wyników nie ma znaczenia semantycznego dla prawdziwości faktów.

149) (Python) Wskaż *nieprawdziwe* twierdzenia o unifikacji.

■ Unifikacja ignoruje nazwy predykatów — liczą się tylko argumenty.

■ Unifikacja zawsze losowo wiąże tę samą zmienną z różnymi stałymi.

■ Unifikacja wymaga, by fakt zawierał zmienne tak jak wzorzec.

■ Unifikacja jest możliwa tylko, gdy arności się różnią.

■ Unifikacja zwraca listę wszystkich faktów, nie substytucję.

150) (Python) Po nasyceniu bazy z regułą `(grandparent)` które zapytania zwrócą dziadków Oli?

☐ `(parent(X,'ola'))`.

☐ `(grandparent('ola','X'))`.

■ `(grandparent(None,'ola'))` w implementacji z wildcardem `None`.

☐ `(grandparent('ola',X))`.

■ `(grandparent(X,'ola'))`.

151) (Python) Które stwierdzenia dot. *reguł bazowych* są poprawne?

■ Może służyć jako punkt startowy dla reguł rekurencyjnych.

■ Zwiększa zbiór zmaterializowanych faktów jeszcze przed wnioskowaniem.

■ Reguła z pustym ciałem jest równoważna faktowi.

■ Może współistnieć z regułami nierekurencyjnymi i rekurencyjnymi.

■ Nie wymaga unifikacji ciała przy wyprowadzaniu głowy.

152) (Python) Wybierz *niepoprawne* twierdzenia o łańcuchowaniu w przód.

- Forward chaining zawsze wymaga negacji w ciele reguł.
- Forward chaining działa tylko dla grafów bez cykli.
- Forward chaining usuwa stare fakty po dodaniu nowych.
- Forward chaining kończy się tylko po stałej liczbie iteracji niezależnie od danych.
- Forward chaining zaczyna od celu i rozkłada go na podcele.

153) (Python) Które zdania o *duplikatach faktów* są prawdziwe?

- Przechowywanie faktów w zbiorze eliminuje duplikaty automatycznie.
- ☐ Duplikaty są wymagane do rekurencji.
- ☐ Duplikaty pozwalają na negację przez niespełnienie.
- ☐ Duplikaty są konieczne, aby wnioskowanie działało.
- ☐ Duplikaty zapobiegają nasyceniu.

154) (Python) Czym jest *atom* w reprezentacji wiedzy wykorzystywanej w regułach Horna?

- ☐ Wynikiem funkcji kosztu używanym w uczeniu.
- ☐ Instrukcją `if-else` w Pythonie.
- ☐ Wyłącznie węzłem w grafie bez etykiety relacji.
- ☐ Dowolnym napisem bez struktury argumentów.
- Zastosowaniem predykatu do uporządkowanej listy termów, np. `parent('anna','eva')`.

155) (Python) Które z poniższych stwierdzeń o *regułach z dwiema przesłankami* jest poprawne?

- Wymagają znalezienia *wspólnej* substytucji spełniającej oba atomy ciała jednocześnie.
- ☐ Zawsze wymagają negacji jednej z przesłanek.
- ☐ Ich zastosowanie nie może dodać nowych faktów.
- ☐ Można zadowolić się dopasowaniem tylko pierwszej przesłanki.
- ☐ Są równoważne pojedynczej przesłance z OR.

156) (Python) Wskaż różnicę między *forward* i *backward chaining*.

- ☐ Forward używa negacji, Backward nie.
- ☐ Forward: od celu do danych; Backward: od danych do losowych faktów.
- ☐ Backward wymaga grafów bez cykli, a Forward nie działa na grafach.
- Forward: od danych do wniosków; Backward: od celu do przesłanek i podcelów.
- ☐ Backward nie korzysta z unifikacji.

157) (Python) Dla relacji `ancestor` wskaż poprawny zestaw reguł.

- `ancestor(X,Z) :- parent(X,Z)` oraz `ancestor(X,Z) :- parent(X,Y), ancestor(Y,Z)`.
- ☐ `ancestor(X,Z) :- grandparent(X,Z)` bez żadnych innych reguł.
- ☐ `ancestor(X,X) :-` jako jedyna reguła.
- ☐ `ancestor(X,Z) :- ancestor(X,Z)` oraz `parent(X,Y) :- ancestor(Y,Z)`.
- ☐ `ancestor(X,Z) :- parent(Z,X)` (odwrócona relacja).

158) (Python) Które z poniższych zdań o *projektowaniu predykatów i arności* są rozsądne?

- ☐ Argumenty nie mają kolejności — można je dowolnie permutować w faktach.
- ☐ Ten sam predykat może znaczyć coś innego w każdej regule.
- ☐ Arność można zmieniać w każdej regule bez konsekwencji.
- Nazwy predykatów powinny odzwierciedlać relację (np. `parent`, `edge`).
- Spójna arność predykatu w całej bazie jest kluczowa dla unifikacji.

159) (Python) W łańcuchowaniu w przód (*forward chaining*):

- ☐ Losowo generujemy fakty bez zastosowania reguł.
- Zaczynamy od faktów i iteracyjnie wyprowadzamy nowe fakty przez stosowanie reguł aż do nasycenia.
- ☐ Zaczynamy od celu i cofamy się do przesłanek.
- ☐ Wymagamy negacji w ciele każdej reguły.

☐ Usuujemy stare fakty, jeśli pojawiają się nowe.

160) Wybierz *falszywe* stwierdzenia o problemach wieloetapowych.

- ☐ Zazwyczaj definiuje się funkcję kosztu ścieżki.
- ☐ Mogą być rozwiązywane metodami informowanymi lub nieinformowanymi.
- ☐ Wymagają specyfikacji stanu początkowego i testu celu.
- ☐ Mogą wymagać pamiętania stanów odwiedzonych, by uniknąć pętli.
- ☐ Często operują na przestrzeni stanów o dużym czynniku rozgałęzienia.

161) Wskaż poprawne stwierdzenia o *Przeszukiwaniu iteracyjnym w głąb*.

- Powtarza eksplorację górnych poziomów, ale narzut czasowy jest zwykle niewielki.
- Składa się z sekwencji wywołań przeszukiwania z ograniczeniem głębokości z narastającym limitem.
- Nie wymaga dodatkowej pamięci na szerokie poziomy jak przeszukiwanie wszerz.
- Jest zupełne na grafach skończonych i znajduje najpłytsze rozwiązanie przy równych kosztach.
- Może być nieoptymalne dla zróżnicowanych kosztów krawędzi w porównaniu z przeszukiwaniem z kosztami.

162) W problemach niedeterministycznych, które modyfikacje wyszukiwania są sensowne?

- Rozszerzenie testów celu o weryfikację wszystkich gałęzi planu.
- ☐ Usuwanie efektów działań z definicji operatorów.
- Traktowanie węzłów AND jako konieczności zaspokojenia wszystkich możliwych wyników akcji.
- ☐ Ignorowanie rozgałęzień wyników w ekspansji.
- ☐ Zastąpienie grafu stanów listą losowych akcji.

163) Czym w kontekście AI jest *problem deterministyczny* przeszukiwania?

- ☐ Zawsze jednocześnie deterministyczny i niedeterministyczny.
- ☐ Taki, w którym wyniki działania są losowe i nieprzewidywalne.
- Taki, w którym zastosowanie operatora w danym stanie prowadzi do jednego, z góry określonego stanu następnego.
- ☐ Taki, który wymaga heurystyki, aby określić następny stan.
- ☐ Taki, w którym nie ma operatorów ani stanów.

164) Które stwierdzenia o *czynniku rozgałęzienia* i *głębokości* rozwiązania są sensowne?

- ☐ Głębokość celu maleje, gdy zwiększymy koszt krawędzi.
- ☐ Czynniki rozgałęzienia dotyczy tylko algorytmów informowanych.
- Wysoki czynnik rozgałęzienia i duża głębokość celu zwiększają złożoność czasową/pamięciową przeszukiwania.
- ☐ Głębokość celu jest równa 1 dla każdego problemu przeszukiwania.
- ☐ Czynniki rozgałęzienia nie wpływają na liczbę generowanych węzłów.

165) Algorytm iteracyjnego przeszukiwania z heurystyką – na czym polega aktualizacja progu i kiedy algorytm kończy?

- ☐ Próg to wyłącznie  $h(\text{start})$ ; kończy, gdy wykorzysta całą pamięć.
- ☐ Próg zmniejsza się w kolejnych iteracjach, aby przyspieszyć wyszukiwanie.
- ☐ Algorytm kończy dopiero po odwiedzeniu wszystkich stanów o  $f$  mniejszych od progu maksymalnego.
- Próg początkowy to  $f(\text{start})$ ; w każdej iteracji nowy próg to najmniejsza wartość  $f$  przekraczająca bieżący próg; algorytm kończy, gdy cel zostanie znaleziony w granicach progu.
- ☐ Próg to liczba węzłów w kolejce; rośnie o stałą liczbę w każdej iteracji.

166) Które stwierdzenia o *kolejkach danych* w strategiach są poprawne?

- Przeszukiwanie z kosztami używa kolejki priorytetowej porządkowanej kosztem ścieżki.
- ☐ Przeszukiwanie z ograniczeniem głębokości używa wyłącznie tablicy mieszającej.
- Przeszukiwanie wszerz używa kolejki FIFO do porządku rozwijania.
- ☐ Przeszukiwanie iteracyjne w głąb wymaga tablicy heurystycznej.
- Przeszukiwanie w głąb używa stosu (LIFO) lub rekurencji.

167) Które opisy *preconditions* i *effects* w STRIPS są poprawne?

- Efekty dodawane dopisują literale do stanu wynikowego, a efekty usuwane je wykluczają.
- ☐ Warunki wstępne mogą być zawsze zastąpione losową heurystyką.
- Warunki wstępne akcji muszą być prawdziwe w stanie wejściowym, aby akcja była wykonalna.

- ☐ Warunki wstępne są ignorowane, jeśli cel zawiera te same literale.
- ☐ Efekty usuwane w STRIPS oznaczają „odłóż wykonanie akcji do końca planu”.

168) Dopasuj problem do strategii, która jest naturalnym pierwszym wyborem.

- Mapa o równych kosztach przejść i cel na małej głębokości → Przeszukiwanie wszerek.
- ☐ Różne, dodatnie koszty krawędzi, szukamy najtańszego rozwiązania → Przeszukiwanie wszerek.
- Głęboka przestrzeń z rzadkimi rozwiązaniami, ograniczona pamięć → Przeszukiwanie w głąb lub z ograniczeniem głębokości.
- ☐ Wysokie, zróżnicowane koszty i cykle → Przeszukiwanie iteracyjne w głąb bez zmian.
- ☐ Nieznana głębokość, brak heurystyki, chcemy najpłytsze rozwiązanie przy małej pamięci → Czyste przeszukiwanie w głąb.

169) Checklista: kiedy *STRIPS* jest naturalnym wyborem?

- ☐ Wymagana jest jawna optymalizacja nad rozkładami wyników akcji.
- ☐ Dominują plany probabilistyczne maksymalizujące oczekiwaną nagrodę.
- Świat binarnych faktów, deterministyczne akcje, cel opisany literami, brak potrzeby planu warunkowego.
- ☐ Domena wymaga wyłącznie metodycznych dekompozycji zadań wysokopoziomowych.
- ☐ Silny niedeterminizm i wymóg planów warunkowych.

170) Które stwierdzenia o *Przeszukiwaniu z ograniczeniem głębokości* są prawdziwe?

- ☐ Zużywa więcej pamięci niż przeszukiwanie wszerek.
- Może przegapić istniejące rozwiązanie, jeśli znajduje się głębiej niż ustawiony limit.
- ☐ Zawsze gwarantuje optymalne rozwiązanie niezależnie od limitu.
- Może zapobiec utknięciu w nieskończonej gałęzi charakterystycznej dla czystego przeszukiwania w głąb.
- ☐ Wymaga równych kosztów kroków, by działać.

171) Checklista doboru algorytmu planowania – które kryteria są kluczowe?

- Wymagana postać planu: sekwencja liniowa, plan równoległy, plan warunkowy/polityka.
- Deterministyczność vs. niedeterministyczność / probabilistyka wyników akcji.
- Wymagania co do optymalności kosztu vs. wystarczalność planu wykonalnego.
- Skala przestrzeni stanów i ograniczenia zasobów (czas, pamięć).
- Dostępność i jakość wiedzy domenowej.

172) Planowanie probabilistyczne: które kryteria optymalności mają sens?

- ☐ Minimalizacja długości planu wyłącznie przez usunięcie preconditions.
- Maksymalizacja oczekiwanej użyteczności przy danych rozkładach wyników.
- ☐ Wymuszenie deterministycznych wyników przez ignorowanie rozkładów.
- ☐ Minimalizacja liczby mutex w grafie planowania.
- Maksymalizacja prawdopodobieństwa osiągnięcia celu przy limicie zasobów.

173) W problemie deterministycznym *graf AND-OR* jest:

- ☐ Konieczny, bo każda akcja ma nieskończenie wiele wyników.
- ☐ Służy wyłącznie do przeszukiwania wszerek w grafach ważonych.
- ☐ To mechanizm do wyznaczania heurystyki admisyjnej.
- ☐ Oznacza, że węzły OR to stany, a AND to heurystyki.
- Zazwyczaj zbędny — wystarcza zwykle drzewo/graf stanów, bo wyniki akcji nie rozgałęziają się na alternatywne światy.

174) Planowanie probabilistyczne vs. niedeterministyczne – dopasuj cele.

- Probabilistyczne → polityka zależna od stanu i rozkładów przejść.
- Probabilistyczne → maksymalizacja oczekiwanej użyteczności / prawdopodobieństwa sukcesu.
- Niedeterministyczne → istnienie strategii osiągnięcia celu dla wszystkich możliwych wyników akcji (plan warunkowy).
- Niedeterministyczne → plan w postaci drzewa z gałęziami na wyniki akcji.
- Probabilistyczne → możliwość kompromisu ryzyko–nagroda przez funkcję użyteczności.

175) Kiedy *Przeszukiwanie z kosztami* i *Przeszukiwanie wszerek* zachowują się identycznie?

- ☐ Nigdy — to różne metody i zawsze dadzą inne drzewo wyszukiwania.
- ☐ Gdy koszty kroków są losowe, ale średnio równe.
- Gdy wszystkie koszty kroków są równe i nieujemne.

- ☐ Gdy w grafie są ujemne koszty krawędzi.
- ☐ Gdy graf jest drzewem binarnym niezależnie od kosztów.

176) Które stwierdzenia o *planie równoległym* w GraphPlan są prawdziwe?

- ☐ W GraphPlan zawsze wykonuje się dokładnie jedną akcję na poziom.
- ☐ Równoległość wymaga, aby akcje miały identyczne preconditions.
- ☐ Jeśli dwie akcje dotyczą rozłącznych faktów, to z definicji są mutex.
- Akcje na tym samym poziomie mogą być wykonywane równolegle, jeśli nie są mutex.
- Ekstrakcja planu wybiera na danym poziomie zbiór akcji, które wspólnie spełniają cele i nie konfliktują.

177) Które twierdzenia o *planie* w problemach niedeterministycznych są poprawne?

- ☐ W niedeterministycznych problemach nie definiuje się planów.
- Realizuje się go jako plan warunkowy: jeśli wystąpi wynik A, wykonaj akcję X; w przeciwnym razie akcję Y.
- Plan może mieć postać drzewa decyzji zawierającego rozgałęzienia zależne od wyniku akcji.
- ☐ Plan jest zawsze jedną listą akcji bez odchyleń.
- ☐ Plany warunkowe są możliwe tylko w problemach jednoetapowych.

178) Wskaż przykłady *jednoetapowych* problemów decyzyjnych.

- ☐ Synteza sekwencji operacji w STRIPS.
- Wybór jednego z gotowych planów na podstawie funkcji oceny.
- ☐ Planowanie kolejności zadań z ograniczeniami precedencji.
- Wybór następnego punktu celu spośród kandydatów w nawigacji z automatycznym wykonaniem przez inny moduł.
- ☐ Układanie ścieżki od startu do celu w labiryncie.

179) Wskaż pary, które opisują poprawne dopasowanie:

- STRIPS → preconditions / effects.
- HTN → dekompozycja zadań złożonych do prymitywnych.
- Planowanie niedeterministyczne → plany warunkowe (strategiczne) na wszystkie możliwe wyniki akcji.
- Planowanie probabilistyczne → modele z niepewnością wyników, np. procesy decyzyjne Markowa.
- GraphPlan → poziomy stanów i akcji oraz relacje mutex.

180) Jakie warunki muszą zachodzić, aby *przeszukiwanie wszerek* było zupełne i optymalne w problemie deterministycznym?

- ☐ Zupełny tylko w drzewach binarnych; optymalny zawsze.
- ☐ Optymalny tylko przy rosnących kosztach wraz z głębokością.
- ☐ Zupełny wyłącznie z heurystyką; optymalny przy kosztach ujemnych.
- ☐ Nigdy nie jest zupełny ani optymalny.
- Zupełny w grafach skończonych (z wykrywaniem odwiedzin); optymalny, gdy wszystkie koszty kroków są równe.

181) Jakie role pełnią *koszt ścieżki* ( $g(n)$ ), *heurystyka* ( $h(n)$ ) i *funkcja oceny* ( $f(n)$ ) w wyszukiwaniu informowanym?

- Admisyjność heurystyki oznacza, że ( $h(n)$ ) nigdy nie przeszacowuje rzeczywistego kosztu do celu.
- ( $f(n)$ ) łączy ( $g(n)$ ) i ( $h(n)$ ) do porządkowania rozwijanych węzłów.
- Dobra heurystyka może znacząco ograniczyć liczbę eksplorowanych węzłów.
- ( $h(n)$ ) szacuje koszt od ( $n$ ) do celu (na podstawie wiedzy domenowej).
- ( $g(n)$ ) sumuje koszty kroków od startu do węzła ( $n$ ).

182) Planowanie hierarchiczne (*Hierarchical Task Networks*, HTN) – wskaż trafne stwierdzenia.

- ☐ HTN nie dopuszcza żadnej hierarchii – wszystkie zadania są równorzędne.
- Wiedza domenowa (metody) kieruje poszukiwaniem planu, ograniczając przestrzeń rozwiązań.
- Zadania złożone są dekomponowane do zadań prostych (prymitywnych) według reguł/metod domenowych.
- ☐ HTN wymaga, aby wszystkie akcje były odwracalne.
- ☐ HTN zakłada brak wiedzy domenowej i losową dekompozycję zadań.

183) Jakie są typowe reprezentacje stanu i operatorów w przestrzeni stanów?

- Koszt ścieżki jako suma kosztów zastosowanych operatorów.
- Test celu jako predykat nad stanem zwracający prawdę/fałsz.
- Operator jako funkcja/przejsicie: stan → nowy stan, opcjonalnie z kosztem.

- Stan jako krotka/rekord opisujący istotne zmienne problemu.
- Graf stanów jawny (predefiniowany) lub generowany „na żądanie” przez funkcję sukcesorów.

184) Które elementy specyfikacji problemu są wspólne dla deterministycznych i niedeterministycznych problemów wieloetapowych?

- Predykat celu lub zbiór stanów docelowych.
- Definicja przestrzeni stanów i funkcji sukcesorów (różniaca się charakterem wyniku).
- Stan początkowy.
- Zbiór operatorów/działań wraz z ich pre-/post-warunkami (efektami).
- Opcjonalna funkcja kosztu kroków/ścieżki.

185) Wskaż poprawne różnice między *Przeszukiwaniem iteracyjnym w głąb* a *Przeszukiwaniem wszerz*.

- ☐ Przeszukiwanie wszerz wymaga znajomości limitu głębokości.
- ☐ Przeszukiwanie iteracyjne w głąb potrzebuje heurystyki domenowej.
- Przeszukiwanie iteracyjne w głąb zużywa pamięć podobną do przeszukiwania w głąb, a mimo to znajduje najpłytsze rozwiązanie.
- ☐ Przeszukiwanie iteracyjne w głąb jest optymalne dla zróżnicowanych kosztów krawędzi.
- Przeszukiwanie wszerz przechowuje całą bieżącą warstwę węzłów i zwykle zużywa znacznie więcej pamięci.

186) Które stwierdzenia poprawnie odróżniają *wyszukiwanie nieinformowane* od *informowanego*?

- ☐ Nieinformowane zawsze znajduje rozwiązanie optymalne dla dowolnych kosztów.
- ☐ Heurystyka to gwarantowana dokładna odległość do celu.
- Nieinformowane nie używa wiedzy o problemie poza strukturą przestrzeni stanów.
- ☐ Informowane nie eksploruje w ogóle gałęzi nieoptymalnych — nie rozwija żadnych węzłów pobocznych.
- Informowane (np. A\*) wykorzystuje heurystykę szacującą „odległość” do celu.

187) Które stwierdzenia o *stanach terminalnych* (celowych) są poprawne?

- W problemach niedeterministycznych plan warunkowy musi doprowadzić do stanu terminalnego niezależnie od rozgałęzienia wyników.
- Zbiór stanów terminalnych może być wieloelementowy (nie jeden stan).
- W problemach jednoetapowych stan terminalny jest osiągany natychmiast po podjęciu decyzji.
- Stan terminalny może mieć przypisany koszt/zwrot użyty do oceny planu.
- Stan spełnia test celu — plan może się zakończyć sukcesem.

188) W problemie *niedeterministycznym jednoetapowym* wybór akcji powinien:

- ☐ Ignorować wyniki, bo i tak nie ma dalszych kroków.
- ☐ Zawsze sprowadzać się do losowania akcji.
- Uwzględniać możliwe wyniki akcji i ich wpływ na osiągnięcie celu/ograniczeń.
- Może wymagać reguł rozstrzygania remisów lub preferencji ryzyka.
- ☐ Wymagać drzewa AND-OR o głębokości większej niż 1 z definicji.

189) Które zdania o kosztach i własnościach algorytmów należy rozważyć przy doborze metody?

- Koszt obliczeniowy samej heurystyki w relacji do oszczędności eksploracji.
- Zapotrzebowanie pamięciowe.
- Stabilność i wpływ polityki ponownych odwiedzin na poprawność i wydajność.
- Optymalność (czy znajdzie rozwiązanie o minimalnym koszcie).
- Zupełność (czy znajdzie rozwiązanie, jeśli istnieje).

190) Cechy dobrej heurystyki – wskaż prawidłowe stwierdzenia.

- ☐ Zawsze powinna przeszacowywać, aby szybciej dojść do celu.
- Jest informatywna, czyli dobrze różnicuje stany bliższe i dalsze od celu.
- W algorytmie A\* pożądana jest dopuszczalność (nie przeszacowuje kosztu do celu).
- ☐ Musi być identyczna dla wszystkich stanów, aby zachować spójność.
- Jest obliczeniowo tania względem kosztu rozwinięcia węzła.

191) Które zdania o *zakończeniu algorytmu* są prawdziwe?

- ☐ Przeszukiwanie w głąb kończy się po odwiedzeniu całej przestrzeni, nawet jeśli cel znaleziono wcześniej.
- ☐ Przeszukiwanie iteracyjne w głąb kończy dopiero po wyczerpaniu wszystkich limitów niezależnie od znalezienia celu.
- ☐ Przeszukiwanie z kosztami musi najpierw odwiedzić wszystkie węzły o mniejszej głębokości niż cel.



- Przeszukiwanie wszerek i z kosztami kończąc, gdy pierwszy raz zdejmowany z kolejki węzeł jest celem (przy standardowych warunkach).
- Przeszukiwanie w głąb z limitem kończy, gdy osiągnie limit lub znajdzie cel.

192) Wskaż *niepoprawne* użycia strategii.

- Szukamy najtańszej ścieżki przy silnie zróżnicowanych kosztach → użycie przeszukiwania w głąb, bo „zużywa mniej pamięci”.
- Nieznana głębokość celu, ograniczona pamięć → przeszukiwanie wszerek zamiast przeszukiwania iteracyjnego w głąb.
- ☐ Dodatkowo wagi krawędzi i potrzeba optymalności → użycie przeszukiwania z kosztami.
- ☐ Ustalony limit zasobów pamięci → rozważenie przeszukiwania w głąb lub przeszukiwania z ograniczeniem głębokości zamiast przeszukiwania wszerek.
- ☐ Równe koszty i płytki cel → użycie przeszukiwania wszerek.

193) Algorytm iteracyjnego przeszukiwania z heurystyką (IDA\*), gdzie  $g(n)$  to koszt ścieżki od startu, a  $h(n)$  to heurystyka do celu – wskaż poprawny opis.

- ☐ Nie wymaga funkcji  $g(n)$  – używa wyłącznie  $h(n)$ .
- Wykonuje kolejne przeszukiwania w głąb ograniczone progiem na  $f(n) = g(n) + h(n)$ , zwiększając próg w kolejnych iteracjach.
- ☐ Jest to przeszukiwanie wszerek z malejącym limitem głębokości.
- ☐ Stosuje jedynie limit na liczbę odwiedzonych węzłów, nie na  $f(n) = g(n) + h(n)$ .
- ☐ Zastępuje heurystykę dokładnym kosztem dojścia do celu.

194) Które stwierdzenia o dopuszczalności i spójności heurystyki są poprawne?

- Spójność (monotoniczność) heurystyki oznacza, że dla każdego przejścia  $n \rightarrow n'$  zachodzi  $h(n) \leq c(n, n') + h(n')$ .
- ☐ Spójność wymaga, aby  $h(n)$  było stale równe zero.
- Dopuszczalność oznacza, że heurystyka nigdy nie przeszacowuje rzeczywistego kosztu do celu.
- ☐ Dopuszczalność i spójność to dokładnie to samo pojęcie w każdym ustawieniu.
- ☐ Każda dopuszczalna heurystyka jest z definicji niedeterministyczna.

195) Jakie techniki są typowe dla *niedeterministycznych* problemów wieloetapowych?

- Wyszukiwanie w grafach AND-OR i synteza planów warunkowych.
- ☐ Zastąpienie planu polityką losową bez warunków.
- Weryfikacja, że plan osiąga cel dla wszystkich możliwych wyników akcji w danym miejscu.
- ☐ Brak testu celu — niedeterminizm go eliminuje.
- ☐ Zawsze wystarczy algorytm zachłanny bez rozgałęzień.

196) Kiedy warto wybrać algorytm iteracyjnego przeszukiwania z heurystyką zamiast algorytmu A\*?

- ☐ Gdy nie posiadamy żadnej heurystyki i chcemy polegać na kosztach wyłącznie.
- ☐ Gdy mamy obfitość pamięci i zależy nam na minimalizacji czasu bez powtarzania eksploracji.
- ☐ Gdy koszty krawędzi są ujemne i często tworzą cykle ujemne.
- ☐ Gdy potrzebujemy równoległego przeszukiwania wszerek.
- Gdy ograniczeniem jest pamięć, a dopuszczalna heurystyka jest dostępna i wciąż pozwala na sensowne progi.

197) Warunki optymalności algorytmu A\* – które są właściwe?

- ☐ A\* jest optymalny tylko wtedy, gdy  $h(n) = 0$  dla wszystkich stanów.
- ☐ A\* nie może korzystać z informacji o kosztach  $g(n)$ .
- ☐ A\* jest optymalny przy dowolnych ujemnych kosztach krawędzi.
- ☐ Optymalność A\* nie zależy od własności heurystyki.
- A\* jest optymalny, jeśli koszty krawędzi są nieujemne, a heurystyka jest dopuszczalna i spójna.

198) Checklista: ograniczenia zasobów a wybór techniki – wskaż sensowne pary.

- Bardzo ograniczona pamięć, plan liniowy, deterministyczny → algorytmy o małym śladzie pamięci.
- ☐ Silny niedeterminizm i potrzeba planu warunkowego → czysty STRIPS bez rozszerzeń.
- ☐ Wysoka równoległość akcji i konflikty → ignorowanie mutex, bo spowalniają.
- Bogata wiedza proceduralna o domenę → planowanie hierarchiczne (HTN).
- ☐ Duże grafy i potrzeba planów równoległych → unikanie GraphPlan, bo nie obsługuje równoległości.

199) Które stwierdzenia o *Przeszukiwaniu w głąb* są prawdziwe?

- ☐ Wymaga znanej głębokości rozwiązania z góry.
- Zużywa relatywnie mało pamięci w porównaniu z przeszukiwaniem wszerek.

- Może wpaść w nieskończoną gałąź, jeśli nie zastosuje się limitu głębokości lub wykrywania cykli.
- ☐ Gwarantuje optymalne rozwiązanie dla dowolnych kosztów.
- ☐ Zawsze znajduje najpłytsze rozwiązanie przed głębszymi.

200) Dla którego typu problemu *deterministyczny, wieloetapowy* czy *niedeterministyczny, wieloetapowy* klasyczne  $A^*$  ( $f=g+h$ ) jest naturalnym wyborem bez modyfikacji?

- Deterministyczny, wieloetapowy.
- ☐ Deterministyczny, jednoetapowy ( $A^*$  wymaga drzew AND-OR).
- ☐ Żaden —  $A^*$  nie dotyczy wyszukiwania ścieżek.
- ☐ Oba typy identycznie, bo niedeterminizm nie wpływa na algorytm.
- ☐ Niedeterministyczny, wieloetapowy ( $A^*$  bez zmian generuje od razu plan warunkowy).

201) (Python) Które cechy planu warto raportować w projektach inżynierskich?

- ☐ Wyłącznie identyfikator procesu w systemie operacyjnym.
- Odporność na zmiany (np. alternatywne gałęzie, jeśli krawędź się „psuje”).
- ☐ Ilość losowań generatora liczb pseudolosowych.
- ☐ Liczba rdzeni CPU komputera trenującego heurystykę.
- Całkowity koszt planu oraz liczba akcji.

202) (Python) Jak radzić sobie z wieloma najkrótszymi ścieżkami o tym samym koszcie?

- Raportować deterministyczny wybór przez ustaloną kolejność sąsiadów.
- Wprowadzić tie-breaking w kolejce (np. preferować mniejsze  $h$  lub leksykograficznie).
- Zdefiniować dodatkowe kryterium (np. minimalna liczba kroków przy równym koszcie).
- Akceptować dowolną z równokosztowych ścieżek jako poprawną.
- Zbierać wszystkie poprzedniki o tym samym koszcie i odtwarzać wiele ścieżek.

203) (Python) Które reprezentacje grafu są typowe w implementacjach algorytmów ścieżki?

- ☐ Losowa permutacja węzłów bez informacji o krawędziach.
- ☐ Macierz jednostkowa zamiast wag krawędzi.
- Lista sąsiedztwa: dla węzła trzymamy listę (sąsiad, koszt).
- Słownik mapujący węzeł na listę krotek ( $v$ , waga).
- ☐ Pojedyncza liczba całkowita jako „graf”.

204) (Python) Które zdania o *warunku stopu* dla algorytmu Dijkstry są poprawne?

- Można odtworzyć ścieżkę do celu.
- ☐ Warunek stopu działa tylko w grafach bez wag.
- ☐ Trzeba rozwinąć wszystkie węzły grafu, nawet po wyjściu celu.
- ☐ Trzeba dodatkowo sprawdzić, czy istnieje krótsza ścieżka z ujemną wagą.
- Gdy zdejmujemy z kolejki cel, jego koszt  $g(\text{goal})$  jest już optymalny.

205) (Python) Wskaż *falszywe* zdania o rekonstrukcji ścieżki po zakończeniu wyszukiwania algorytmu Dijkstry.

- Kierunek odtwarzania musi być od startu do celu, w przeciwnym razie ścieżka jest błędna.
- ☐ Mapa poprzedników jest aktualizowana przy poprawie  $g(\text{sąsiada})$ .
- Nie potrzebujemy mapy poprzedników; ścieżkę zawsze zgadniemy po kosztach.
- ☐ Typowo odtwarzamy wstecz: od celu idąc po poprzednikach do startu, a następnie odwracamy listę.
- ☐ Brak poprawnych poprzedników oznacza brak ścieżki.

206) (Python) Kiedy algorytm Dijkstry gwarantuje optymalność znalezionej ścieżki?

- ☐ Gdy kolejka priorytetowa wybiera zawsze największy koszt.
- Gdy wszystkie wagi krawędzi są nieujemne.
- ☐ Wyłącznie w grafach skierowanych acyklicznych z wagami ujemnymi.
- ☐ Gdy istnieje przynajmniej jedna krawędź o ujemnej wadze.
- ☐ Tylko wtedy, gdy graf jest drzewem binarnym.

207) (Python) Wskaż *niepoprawne* stwierdzenia o odległościach w siatce 4-kierunkowej.

- Wartość heurystyki powinna być  $\geq$  rzeczywistego kosztu, by przyspieszyć  $A^*$ .

- ☐ Jeśli dozwolone są tylko ruchy góra/dół/lewo/prawo, Manhattan nie przeszacowuje najkrótszej liczby kroków.
- Manhattan przeszacowuje koszt, gdy ruch na ukos jest zabroniony i koszty są nieujemne.
- ☐ W siatce 4-kierunkowej Manhattan zwykle nie przeszacowuje.
- Odległość euklidesowa jest zawsze  $\leq$  odległości Manhattan, dlatego jest zawsze lepszą heurystyką.

208) (Python) Które strategie rozwiążą problem najkrótszej ścieżki w grafie z nieujemnymi wagami?

- ☐ Losowe chodzenie po grafie do czasu trafienia w cel.
- A\* z heurystyką nie przeszacowującą.
- ☐ Przeszukiwanie wszerek bez modyfikacji w grafie o wagach  $> 1$ .
- ☐ Algorytm Bellmana-Forda tylko dla dodatnich wag.
- Dijkstra (z kolejką priorytetową).

209) (Python) Kiedy *odległość Manhattan* jest naturalną heurystyką?

- W siatkach z ruchem 4-kierunkowym i nieujemnymi kosztami pól.
- ☐ Gdy dozwolone są tylko skoki rycerza jak w szachach.
- ☐ Gdy koszt wejścia na pole jest zawsze ujemny.
- ☐ W pełnych grafach, gdzie koszt każdej krawędzi to -1.
- ☐ W przestrzeni ciągłej z metryką euklidesową i lotami na ukos bez kosztu.

210) Softmax i entropia krzyżowa. Logity  $\mathbf{z} = (1, 0, 0)$ ; prawdopodobieństwa Softmax  $p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$ ;  $\frac{\partial L}{\partial z_i} = p_i - y_i$ ; prawdziwa etykieta to pierwsza klasa  $\mathbf{y} = (1, 0, 0)$ . Podaj przybliżony gradient  $dL/d\mathbf{z}$ .

- ☐  $(0, 0, 0)$ .
- ☐  $(-0.5, 0.5, 0)$ .
- Około  $(-0.424, 0.212, 0.212)$ .
- ☐ Około  $(0.424, -0.212, -0.212)$ .
- ☐  $(-1, 1, 0)$ .

211) Co jest typowym przykładem *uczenia nienadzorowanego* dla sieci neuronowych?

- Wykrywanie anomalii przez odchylenia od typowych wzorców.
- Uczenie rozkładów danych do generowania przykładów.
- Ustalanie reprezentacji poprzez maksymalizację podobieństwa pozytywnych par.
- Autoenkoder uczony do rekonstrukcji wejścia.
- Klasteryzacja reprezentacji wewnętrznych.

212) Co oznacza, że sieć neuronowa jest *warstwowa*?

- ☐ Każdy neuron jest połączony tylko z samym sobą.
- ☐ Warstwy są losowo pomijane w trakcie przewidywania.
- Przetwarzanie odbywa się etapami: wyjścia jednej warstwy stają się wejściami kolejnej.
- ☐ Warstwy służą jedynie do przechowywania hiperparametrów.
- ☐ Kolejność warstw nie ma wpływu na wynik.

213) Jakie *zjawiska niepożądane* związane z funkcjami aktywacji należy brać pod uwagę?

- Eksplodujący gradient przy nieprawidłowej skali parametrów.
- Zanikający gradient w obszarach nasycenia.
- Niestabilność numeryczna przy skrajnych wartościach wejścia.
- Przesunięcie średniej aktywacji utrudniające uczenie.
- Martwe neurony w funkcjach, które wyzerowują wyjście dla części zakresu.

214) Dwuwarstwowy model liniowy bez nieliniowości:  $\hat{\mathbf{y}} = \mathbf{W}_2 * (\mathbf{W}_1 * \mathbf{x})$  (skalary). Strata:  $L = 0.5 * (\hat{\mathbf{y}} - \mathbf{y})^2$ . Dla  $\mathbf{x}=2$ ,  $\mathbf{W}_1=3$ ,  $\mathbf{W}_2=4$ ,  $\mathbf{y}=10$  oblicz  $dL/d\mathbf{W}_1$  i  $dL/d\mathbf{W}_2$ .

- ☐  $dL/d\mathbf{W}_1 = 84$ ,  $dL/d\mathbf{W}_2 = 112$ .
- ☐  $dL/d\mathbf{W}_1 = 14$ ,  $dL/d\mathbf{W}_2 = 14$ .
- ☐  $dL/d\mathbf{W}_1 = 0$ ,  $dL/d\mathbf{W}_2 = 0$ .
- ☐  $dL/d\mathbf{W}_1 = -112$ ,  $dL/d\mathbf{W}_2 = -84$ .
- $dL/d\mathbf{W}_1 = 112$ ,  $dL/d\mathbf{W}_2 = 84$ .

215) Dlaczego *funkcja prostująca* (ReLU) może prowadzić do zjawiska „martwych neuronów”?

- ☐ Ponieważ powoduje eksplodujący gradient dla wartości bliskich zeru.
- ☐ Ponieważ zawsze saturuje się do wartości jeden dla wszystkich wejść.
- ☒ Ponieważ dla wartości ujemnych zwraca zero, a pochodna wynosi zero, co może uniemożliwić dalszą aktualizację wag dla tych neuronów.
- ☐ Ponieważ wymaga normalizacji do zakresu od zera do jedynki w każdej warstwie.
- ☐ Ponieważ dla wartości dodatnich jest nieliniowa i ma zbyt dużą pochodną dodatnią.

216) Które z poniższych są *przykładami* funkcji aktywacji?

- ☐ Spadek gradientowy z momentum.
- ☐ Optymalizator Adam.
- ☐ Warstwa plotowa dwuwymiarowa (konwolucja 2D).
- ☐ Algorytm sortowania szybkiego (Quicksort).
- ☐ Normalizacja wsadowa (Batch Normalization).

217) Jak *normalizacja lub standaryzacja* wejść może oddziaływać na wybór funkcji aktywacji?

- ☐ Normalizacja sprawia, że każda funkcja aktywacji staje się liniowa.
- ☐ Standaryzacja zawsze zwiększa zasięg saturacji w funkcjach ograniczonych.
- ☒ Utrzymanie wejść w rozsądnym zakresie pozwala unikać nasycenia funkcji ograniczonych i sprzyja przepływowi gradientu.
- ☐ Standaryzacja wymusza, aby każda aktywacja była identyczna w całej sieci, co zastępuje proces uczenia.
- ☐ Normalizacja zawsze eliminuje potrzebę funkcji aktywacji w warstwach ukrytych.

218) Do czego służy *funkcja miękka maksymalizacja* (Softmax) w warstwie wyjściowej?

- ☐ Do wyznaczania odległości euklidesowej między próbkami w partii danych.
- ☐ Do losowego permutowania kolejności próbek w zbiorze testowym.
- ☒ Do przekształcania wektora wyników na rozkład prawdopodobieństwa klas, który sumuje się do jedności.
- ☐ Do binarnego progowania każdej cechy wejściowej niezależnie od pozostałych.
- ☐ Do zastąpienia funkcji straty w uczeniu nadzorowanym.

219) Jak *współczynnik uczenia* wpływa na trening?

- ☐ Nie ma znaczenia — optymalizator zawsze kompensuje dowolną wartość.
- ☐ Powinien być większy dla warstw bliżej wejścia, a mniejszy dla wyjścia w każdym zadaniu bez wyjątku.
- ☒ Zbyt niski spowalnia uczenie i może utknąć w płaskich regionach funkcji straty.
- ☒ Zbyt wysoki może powodować rozbieganie się strat i niestabilność.
- ☒ Harmonogramy (schedulery) mogą dynamicznie zmniejszać współczynnik uczenia w trakcie treningu.

220) Jak w prostym ujęciu zbudowana jest *warstwa* w sieci neuronowej?

- ☐ To pojedynczy neuron bez wag i bez funkcji aktywacji.
- ☒ To zbiór neuronów obliczających równolegle swoje wyjścia na podstawie tych samych wejść, zwykle z własnymi wagami i przesunięciami.
- ☐ To specjalny typ funkcji kosztu bez parametrów.
- ☐ To pamięć masowa przechowująca surowe dane, bez obliczeń.
- ☐ To mechanizm do sortowania przykładów treningowych.

221) Generatywne sieci przeciwstawne (Generative Adversarial Networks) — wybierz prawdziwe stwierdzenia.

- ☐ Wymagają etykiet klas do funkcji straty generatora.
- ☒ Potrafią generować realistyczne obrazy z losowego wektora wejściowego.
- ☐ Nie mają problemów z niestabilnością uczenia.
- ☒ Mogą cierpieć na zapadanie się trybów (mode collapse).
- ☒ Składają się z generatora i dyskryminatora trenowanych w grze o sumie zerowej.

222) Jaką *rolę* pełni *funkcja straty* w uczeniu sieci neuronowych?

- ☐ Służy do serializacji modelu do formatu obrazu.
- ☒ Ilościowo mierzy błąd predykcji względem prawdy odniesienia i kieruje procesem optymalizacji parametrów.
- ☐ Losowo zmienia etykiety, aby zwiększyć różnorodność danych.
- ☐ Zastępuje dane treningowe generowanym szumem.
- ☐ Blokuje aktualizację wag po pierwszej epoce.

223) Które z poniższych są *przykładami* funkcji aktywacji?

- Funkcja tangens hiperboliczny (tanh).
- Funkcja prostująca (ReLU).
- Funkcja wyciekająca prostująca (Leaky ReLU).
- Funkcja miękka plus (Softplus).
- Funkcja sigmoidalna (sigmoid).

224) Które stwierdzenia są błędne?

- Dropout zwiększa liczbę aktywnych neuronów, aby przyspieszyć uczenie.
- Zastosowanie regularyzacji eliminuje konieczność walidacji.
- Regularyzacja L2 zwiększa wartości wag, aby poprawić dopasowanie.
- Dropout musi być taki sam na treningu i na teście, inaczej wyniki są bez sensu.
- Regularyzacja L1 uniemożliwia uzyskanie wag równych zero.

225) Do jakich scenariuszy najlepiej pasują poniższe techniki?

- ☐ Dropout — najlepszy wybór do małych modeli liniowych z jedną wagą.
- ☐ Regularyzacja L1 — zawsze lepsza od L2 w każdej architekturze.
- Regularyzacja L1 — gdy zależy nam na rzadkich rozwiązaniach i interpretowalności (selekcja cech).
- Regularyzacja L2 — gdy chcemy stabilizować uczenie i ograniczać duże wagi bez wyzerowywania ich.
- Dropout — gdy chcemy zmniejszyć ko-adaptacje w warstwach ukrytych dużych sieci.

226) Wskaż *prawidłowy opis* obliczenia w neuronie sztucznym.

- ☐ Neuron zawsze zwraca średnią arytmetyczną wejść bez wag.
- Neuron oblicza sumę ważoną wejść, dodaje przesunięcie i przepuszcza wynik przez nieliniową funkcję aktywacji.
- ☐ Neuron wyjściowy nie może używać funkcji aktywacji.
- ☐ Neuron ignoruje wszystkie wejścia równe zero i zwraca jeden.
- ☐ Neuron losuje wyjście niezależnie od wejść.

227) Jakie *cechy* powinna mieć dobra funkcja aktywacji w warstwach ukrytych?

- ☐ Brak ciągłości i brak pochodnej w każdym punkcie jako wymóg konieczny.
- ☐ Wartości wyjściowe zawsze dodatnie i zawsze równomiernie rozłożone w zakresie od zera do jedynek.
- Wystarczająca nieliniowość oraz łatwo obliczalna pochodna dla uczenia metodą spadku gradientowego.
- Stabilne zachowanie numeryczne oraz możliwość ograniczania problemu zanikającego lub eksplodującego gradientu.
- ☐ Wymóg, aby wyjście było zawsze równe wejściu.

228) Które architektury są szczególnie dostosowane do grafów i sieci relacyjnych?

- ☐ Rekurencyjne sieci jednoznacznie lepsze od grafowych na danych grafowych.
- ☐ Klasyczne perceptrony wielowarstwowe bez informacji o strukturze krawędzi.
- Sieci neuronowe oparte na grafach (Graph Neural Networks).
- Splotowe sieci oparte na grafach (Graph Convolutional Networks).
- Sieci z uwagą oparte na grafach (Graph Attention Networks).

229) Grafowe sieci z uwagą (Graph Attention Networks) — wybierz prawdziwe twierdzenia.

- ☐ Nie nadają się do grafów o zmiennej liczbie sąsiadów.
- Mogą przewyższać proste uśrednianie sąsiedztwa w heterogenicznych grafach.
- ☐ Z definicji zakładają w pełni połączone grafy kompletne.
- Uczą wagi krawędziowe poprzez mechanizm uwagi zależny od par wierzchołków.
- Pozwalają różnicować wkład sąsiadów do agregacji cech.

230) Jakie *wady* może mieć funkcja sigmoidalna w głębokich sieciach?

- Może powodować zanikający gradient w obszarach nasycenia oraz przesuwac średnią aktywacji, co spowalnia uczenie.
- ☐ Uniemożliwia klasyfikację binarną, ponieważ nie jest ograniczona w zakresie.
- ☐ Zawsze daje wyniki ujemne, co utrudnia interpretację prawdopodobieństw.
- ☐ Nie ma pochodnej, więc nie można użyć metod gradientowych.
- ☐ Powoduje zawsze eksplodujący gradient niezależnie od danych i parametrów.

231) Zaznacz wszystkie poprawne stwierdzenia o łączeniu technik.

- ☒ Harmonogram tempa uczenia może ograniczyć przeuczenie nawet przy słabszej karze wag.
- ☒ W praktyce często łączy się Dropout z Regularyzacją L2.
- ☒ Augmentacja danych działa jak regularyzacja, zwiększając różnorodność próbek.
- ☒ Wczesne zatrzymanie może pełnić rolę regularyzacji uzupełniającej.
- ☒ Normalizacja wsadowa może zmniejszać potrzebę silnej regularyzacji wag.

232) Które stwierdzenia o spadku gradientowym stochastycznym (SGD) są prawdziwe?

- ☐ Nie działa dla problemów z dużą liczbą parametrów.
- ☒ Aktualizuje parametry po każdej próbkce lub małej paczce danych, co wprowadza losowy szum w kierunku spadku.
- ☐ Zawsze zbiega szybciej niż wszystkie metody adaptacyjne.
- ☐ Nie wymaga ustawiania tempa uczenia, bo wylicza je automatycznie.
- ☒ Może uciec z płytkich minimów lokalnych dzięki fluktuacjom.

233) Które zdania *błędnie* opisują rolę funkcji aktywacji w warstwie wyjściowej?

- ☐ W klasyfikacji binarnej używa się często funkcji dającej wynik możliwy do interpretacji jako prawdopodobieństwo klasy pozytywnej.
- ☒ W klasyfikacji wieloklasowej lepiej nie normalizować wyników, ponieważ suma prawdopodobieństw nie powinna wynosić jeden.
- ☐ W klasyfikacji wieloklasowej naturalne jest przekształcenie wyników na rozkład prawdopodobieństwa klas.
- ☒ W każdej regresji należy używać funkcji ograniczającej zakres od zera do jedynki.
- ☐ W regresji ciągłej często stosuje się wyjście liniowe bez ograniczenia zakresu.

234) Gradient clipping — po co się go używa?

- ☒ Aby ograniczać zbyt duże wartości gradientów i zapobiegać niestabilnym, zbyt dużym krokom.
- ☐ Wymusza, by gradient zawsze był równy zeru poza ostatnią warstwą.
- ☒ Jest szczególnie pomocny w modelach sekwencyjnych, gdzie mogą występować eksplodujące gradienty.
- ☐ Zastępuje potrzebę użycia tempa uczenia.
- ☐ Służy do redukcji rozmiaru modelu przez „obcinanie” warstw.

235) Gdzie najczęściej spotkamy sieci spłotowe jednowymiarowe?

- ☒ Ekstrakcja cech z szeregów czasowych dla prognozowania.
- ☒ Przetwarzanie sygnałów czasowych, takich jak audio lub dane czujników.
- ☐ Wyłącznie segmentacja trójwymiarowych wolumenów medycznych.
- ☐ Detekcja kolizji w symulacjach fizycznych bez danych.
- ☒ Analiza sekwencji znaków lub słów po odpowiednim zakodowaniu.

236) Neuron z wyciekającą funkcją prostującą:  $z = w \cdot x + b$ ,  $y_{\text{hat}} = \text{LeakyReLU}(z)$  z  $\alpha = 0.1$ ,  $L = 0.5 \cdot (y_{\text{hat}} - y)^2$ . Dla  $x=5$ ,  $w=-0.2$ ,  $b=0$ ,  $y=1$  oblicz  $dL/dw$  i  $dL/db$ .

- ☐  $dL/dw = 0.55$ ,  $dL/db = 0.11$ .
- ☐  $dL/dw = -1.1$ ,  $dL/db = -0.22$ .
- ☒  $dL/dw = -0.55$ ,  $dL/db = -0.11$ .
- ☐  $dL/dw = 0$ ,  $dL/db = 0$ .
- ☐  $dL/dw = -0.11$ ,  $dL/db = -0.55$ .

237) Neuron z sigmoidą i MSE:  $z = w \cdot x + b$ ,  $y_{\text{hat}} = \text{sigmoid}(z)$ ,  $L = 0.5 \cdot (y_{\text{hat}} - y)^2$ . Dla  $x=2$ ,  $w=-1$ ,  $b=0$ ,  $y=0.5$  oblicz  $dL/dw$  i  $dL/db$ . (Użyj:  $\text{sigmoid}(-2) = 0.1192$ , pochodna w tym punkcie 0.105.)

- ☐  $dL/dw = 0$ ,  $dL/db = 0$ .
- ☐  $dL/dw = 0.08$ ,  $dL/db = 0.04$ .
- ☐  $dL/dw = -0.40$ ,  $dL/db = -0.20$ .
- ☐  $dL/dw = -0.16$ ,  $dL/db = -0.08$ .
- ☒  $dL/dw = -0.08$ ,  $dL/db = -0.04$ .

238) Co w prostym ujęciu oznacza *głębokość* sieci neuronowej?

- ☐ Liczbę epok treningowych potrzebnych do zbieżności.
- ☐ Maksymalny rozmiar partii danych używany podczas treningu.
- ☐ Szerokość pojedynczej warstwy wejściowej.
- ☐ Wartość funkcji straty na zbiorze testowym.

■ Liczbę warstw przekształcających sygnał pomiędzy wejściem a wyjściem.

239) Na czym polega różnica między momentum Nesterova a „zwykłym” momentum?

■ W klasycznym momentum gradient liczony jest w bieżącym punkcie bez przesunięcia.

■ Momentum Nesterova wykonuje „spojrzenie do przodu” i oblicza gradient w punkcie przewidywanym przez wektor pędu.

☐ Obie wersje zawsze dają identyczne trajektorie aktualizacji.

☐ Klasyczne momentum wymaga adaptacyjnego tempa uczenia z definicji.

☐ Momentum Nesterova nie używa współczynnika pędu.

240) Momentum: jaki jest jego główny cel?

☐ Zawsze wymuszać malejące tempo uczenia niezależnie od ustawień.

☐ Zastąpić gradient wartościami funkcji straty.

☐ Losowo resetować wagi co epokę, aby uniknąć przeuczenia.

■ Wygładzać kierunek aktualizacji przez akumulację przeszłych gradientów, co przyspiesza ruch w dolinach wzdłuż osi o małej krzywiznie.

☐ Ustawić maksymalny krok równy zero, by zapobiec niestabilności.

241) Dany jest neuron liniowy:  $\hat{y} = w \cdot x + b$ . Funkcja straty:  $L = 0.5 \cdot (\hat{y} - y)^2$ . Dla  $x=3$ ,  $w=2$ ,  $b=1$ ,  $y=5$  oblicz  $dL/dw$  i  $dL/db$ .

☐  $dL/dw = 2$ ,  $dL/db = 6$ .

☐  $dL/dw = 0$ ,  $dL/db = 0$ .

☐  $dL/dw = 3$ ,  $dL/db = 1$ .

■  $dL/dw = 6$ ,  $dL/db = 2$ .

☐  $dL/dw = -6$ ,  $dL/db = -2$ .

242) Neuron z funkcją prostującą:  $\hat{y} = \text{ReLU}(z)$ ,  $z = w \cdot x$ . Strata:  $L = 0.5 \cdot (\hat{y} - y)^2$ . Dla  $x=5$ ,  $w=-1$ ,  $y=1$  oblicz  $dL/dw$ .

■  $dL/dw = 0$ .

☐  $dL/dw = 10$ .

☐  $dL/dw = 1$ .

☐  $dL/dw = -5$ .

☐  $dL/dw = -2$ .

243) Które zastosowania są typowe dla konwolucyjnych sieci neuronowych?

☐ Bezpośrednie dowodzenie twierdzeń matematycznych symbolicznymi regułami.

■ Segmentacja semantyczna i instancyjna.

■ Klasyfikacja obrazów i wykrywanie obiektów.

☐ Rozwiązywanie układów równań liniowych metodą Gaussa bez danych.

■ Super-rozdzielczość i odsumianie obrazów.

244) Po co w neuronie sztucznym stosuje się *funkcję aktywacji*?

☐ Aby zastąpić proces uczenia się losowym szumem.

☐ Aby zagwarantować, że wagi nigdy się nie zmieniają.

☐ Aby wymusić, że wyjście zawsze jest równe wejściu.

■ Aby wprowadzić nieliniowość i umożliwić modelowi aproksymację złożonych, nieliniowych zależności.

☐ Aby zmniejszyć liczbę warstw wymaganą do reprezentacji funkcji liniowych.

245) Kiedy *funkcja wyciekająca prostująca* (Leaky ReLU) bywa lepsza niż funkcja prostująca (ReLU)?

☐ Gdy dążymy do całkowitego wyeliminowania nieliniowości w sieci.

■ Gdy chcemy ograniczyć problem martwych neuronów, dopuszczając małe ujemne nachylenie dla wartości ujemnych.

☐ Gdy chcemy dokładnie odtworzyć funkcję sigmoidalną.

☐ Gdy potrzebujemy funkcji, która zawsze zwraca wartości tylko równe zero albo jeden.

☐ Gdy wymagamy, aby wyjście zawsze było dodatnie niezależnie od wejścia.

246) Neuron z sigmoidą:  $z = w \cdot x + b$ ,  $\hat{y} = \text{sigmoid}(z)$ . Strata:  $L = 0.5 \cdot (\hat{y} - y)^2$ . Dla  $x=1$ ,  $w=0$ ,  $b=0$ ,  $y=1$  oblicz  $dL/dw$  oraz  $dL/db$ . (Użyj faktu:  $\text{sigmoid}(0)=0.5$ , pochodna sigmoidy w 0 to 0.25.)

☐  $dL/dw = 0.125$ ,  $dL/db = 0.125$ .

☐  $dL/dw = -0.5$ ,  $dL/db = -0.5$ .

☐  $dL/dw = -0.25$ ,  $dL/db = -0.25$ .

☒  $dL/dw = -0.125$ ,  $dL/db = -0.125$ .

☐  $dL/dw = 0$ ,  $dL/db = 0$ .

247) Adam — wskaż prawdziwe stwierdzenia.

☒ Stosuje korekcję obciążenia na początku treningu, aby zniwelować efekt zerowej inicjalizacji momentów.

☒ Łączy momentum pierwszego rzędu (średnia gradientów) i RMSProp (średnia kwadratów gradientów).

☐ Nie wymaga doboru tempa uczenia ani żadnych hiperparametrów.

☐ Nie nadaje się do rzadkich gradientów, bo ignoruje skalowanie per parametr.

☐ Zawsze zapewnia lepszą generalizację niż SGD z pędem, niezależnie od zadania.

248) Jak działa technika Dropout w czasie treningu?

☒ Losowo dezaktywuje (zeruje) część aktywacji neuronu w czasie treningu.

☐ Zawsze działa tylko w warstwach wyjściowych, bo w ukrytych jest bezużyteczny.

☐ Zwiększa wartości aktywacji, aby kompensować zanikający gradient.

☒ Wymusza na sieci brak współzależności między konkretnymi neuronami.

☐ W czasie ewaluacji zawsze losowo wyłącza neurony tak samo jak na treningu.

249) Które stwierdzenia *fałszywie* opisują funkcję sigmoidalną?

☒ Zwraca wartości w bardzo szerokim zakresie od minus miliona do miliona.

☒ Nie jest ciągła i posiada nieskończenie wiele nieciągłości.

☒ Jest funkcją ściśle liniową, dlatego nie nadaje się do sieci neuronowych.

☒ Ma wartości wyjściowe tylko w liczbach całkowitych.

☒ Zawsze ma pochodną równą jeden niezależnie od wejścia.

250) Które stwierdzenia dotyczące doboru współczynnika regularyzacji są poprawne?

☐ Najlepszą praktyką jest zawsze ustawienie bardzo dużej wartości, aby zapobiec każdemu przeuczeniu.

☐ Nie ma sensu modyfikować siły regularyzacji, bo model i tak zbiega do tego samego minimum.

☐ Współczynnik regularyzacji powinien być równy zeru, jeśli używamy Dropout.

☒ Zbyt duża kara może prowadzić do niedouczenia nawet przy dużej pojemności modelu.

☒ Wartość współczynnika regularyzacji zwykle dobiera się walidacyjnie lub metodami przeszukiwania siatki.

251) Które z poniższych opisów *błędnie* definiują neuron sztuczny?

☒ Neuron to algorytm sortujący listę liczb niezależnie od wag.

☒ Neuron to generator liczb losowych, który ignoruje wejścia.

☒ Neuron to moduł pamięci masowej bez obliczeń.

☒ Neuron to stała liczba, która nie zależy od danych.

☒ Neuron to wyłącznie funkcja liniowa bez przesunięcia i bez możliwości nieliniowej transformacji.

252) Czym w ogólnym ujęciu są *sieci neuronowe* w sztucznej inteligencji?

☐ To wyłącznie urządzenia fizyczne naśladujące neurony biologiczne bez oprogramowania.

☐ To bazy danych przechowujące wyłącznie statyczne reguły bez procesu uczenia.

☐ To systemy operacyjne do zarządzania pamięcią komputera.

☒ To modele obliczeniowe złożone z połączonych jednostek obliczeniowych (neuronów), które uczą się przekształcać wejścia w wyjścia poprzez dostrajanie wag na podstawie danych.

☐ To algorytmy sortowania liczb całkowitych metodą przez wstawianie.

253) Kiedy *MSE* (*Mean Squared Error*) jest naturalnym wyborem funkcji straty?

☒ W regresji, gdy celem jest przewidywanie wartości ciągłych z karą kwadratową za odchylenie.

☐ W detekcji anomalii zawsze zamiast strat rekonstrukcyjnych.

☐ W problemach rankingowych jako jedyna poprawna funkcja.

☒ Gdy zakładamy rozkład błędu o charakterze gaussowskim z wariancją stałą.

☐ W klasyfikacji wieloklasowej z wyjściem softmax jako wybór domyślny.

254) Oblicz pochodne dla neuronu liniowego:  $y_{\text{hat}} = w \cdot x + b$ ,  $L = 0.5 \cdot (y_{\text{hat}} - y)^2$ . Dla  $x=4$ ,  $w=1.5$ ,  $b=-2$ ,  $y=3$  podaj  $dL/dw$  i  $dL/db$ .

☒  $dL/dw = 4$ ,  $dL/db = 1$ .

☐  $dL/dw = 2$ ,  $dL/db = 2$ .



☐  $dL/dw = 0$ ,  $dL/db = 0$ .

☐  $dL/dw = -4$ ,  $dL/db = -1$ .

☐  $dL/dw = 1$ ,  $dL/db = 4$ .

255) Aktualizacja biasu metodą spadku gradientowego:  $b := b - \text{eta} * dL/db$ . Dla  $b=0.5$ ,  $dL/db = -0.2$ ,  $\text{eta}=0.1$  oblicz nowe  $b$ .

☐  $b = 0.48$ .

☒  $b = 0.52$ .

☐  $b = 0.5$ .

☐  $b = 0.7$ .

☐  $b = 0.3$ .

256) Jak *wybór funkcji aktywacji* w warstwach ukrytych wpływa na *głębokość i zbieżność* uczenia?

☐ Wybór funkcji aktywacji nie ma żadnego wpływu na zbieżność ani na głębokość, bo decyduje wyłącznie rozmiar partii danych.

☐ Funkcje bez pochodnej są preferowane, bo przyspieszają obliczenia gradientu.

☒ Funkcje o lepszym przenoszeniu gradientu umożliwiają skuteczniejsze uczenie głębszych sieci i zmniejszają liczbę koniecznych iteracji.

☐ Każda funkcja aktywacji zapewnia identyczną dynamikę uczenia w każdej architekturze.

☐ Im silniejsze nasycenie funkcji, tym zawsze szybsze i stabilniejsze uczenie.